AI-POWERED DEEPFAKE AND SYNTHETIC MEDIA DETECTION SYSTEM

A MINI PROJECT REPORT

Submitted by

ALAN BENNY

Reg. No: MLM25MCA-2014

To

the APJ Abdul Kalam Technological University
in Partial fulfillment of the requirements for
the award of the degree
of
MASTER OF COMPUTER APPLICATIONS

Under the Guidance of

Ms. ASHWANI VIJAYACHANDRAN
(Assistant Professor, Department of Computer Applications)

DEPARTMENT OF COMPUTER APPLICATIONS
MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR
(Affiliated to APJ Abdul Kalam Technological University)

OCTOBER 2026

MANGALAM COLLEGE OF ENGINEERING

Accredited by NAAC& ISO 9001:2000 Certified Institution

DEPARTMENT OF COMPUTER APPLICATIONS

VISION

To become a centre of excellence in computer applications,competent in the global ecosystem with technical knowledge,innovation with a sense of social commitment.

MISSION

- To serve with state of the art ducation,foster advanced research and cultivate innovation in the field of computer applications.
- To prepare learners with knowledge skills and critical thinking to excel in the technological landscape and contribute positively to society.

Program Educational Objectives

- PEO I :Graduates will possess a solid foundation and in-depth understanding of computer applications and will be equipped to analyze real-world problems, design and create innovative solutions, and effectively manage and maintain these solutions in their professional careers.
- PEO II: Graduates will acquire technological advancements through continued education, lifelong learning and research, thereby making meaningful contributions to the field of computing.
- PEO III: Graduates will cultivate team spirit, leadership, communication skills, ethics, and values, enabling them to apply their understanding of the societal impacts of computer applications effectively.

Program Specific Outcomes

- **PSO I**: Apply advanced technologies through innovations to enhance the efficiency of design and development.
- **PSO II**: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

MAPPING OF PO-PSO-SDG

MAPPING WITH PROGRAM OUTCOMES (POs):

SL. NO	POs ADDRESSED	RELEVANCE TO PROJECT
1	PO1 — Foundation Knowledge	Convolutional network design, weighted probability fusion, discrete cosine transform analysis and Laplacian variance computation were applied directly from mathematics and programming fundamentals to build the detection pipeline.
2	PO2 — Problem Analysis	The failure modes of existing single-model detectors were reviewed and reformulated as a requirement for multi-signal evidence with an explicit uncertainty outcome, driven by what an ordinary user actually needs from a verdict.
3	PO3 — Development of Solutions	A working web platform was designed, implemented and tested end to end, contributing to SDG 9 and SDG 16 by making media-integrity verification freely available rather than restricted to forensic laboratories.
4	PO4 — Modern Tool Usage	TensorFlow, Keras, OpenCV, Django 5, yt-dlp and ReportLab were selected and adapted, and the known limitations of each signal are documented and surfaced to the user instead of being concealed.
5	PO5 — Individual and Teamwork	The project was planned and executed individually under faculty supervision using incremental iterations, with progress tracked through version control and reviewed at regular guide meetings.
6	PO6 — Project Management and Finance	A work breakdown structure covering dataset preparation, model training, web development and testing was scheduled across the semester, and the entire system was delivered using only free and open-source components.
7	PO7 — Ethics	Server-side request forgery protection, upload size and type validation, and a deliberate refusal to overstate confidence were built in so that the tool cannot be casually misused to defame a person on weak evidence.
8	PO8 — Life-long learning	Working with generative-media forensics required continuous study of techniques published well after the prescribed syllabus, and the modular signal design allows new detectors to be added as the threat evolves.

LIST OF PROGRAM OUTCOMES (POs):

PO1 (Foundation Knowledge): Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.

PO2 (Problem Analysis): Identify, review, formulate and analyze problems for primarily focusing on customer requirements using critical thinking frameworks.

PO3 (Development of Solutions): Design, develop and investigate problems with an innovative approach for solutions incorporating ESG/SDG goals.

PO4 (Modern Tool Usage): Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO5 (Individual and Teamwork): Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.

PO6 (Project Management and Finance): Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.

PO7 (Ethics): Commit to professional ethics in managing software projects with financial aspects. Learn to use new technologies for cyber security and insulate customers from malware.

PO8 (Life-long learning): Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

MAPPING WITH PROGRAM SPECIFIC OUTCOMES (PSOs):

SL. NO	PSOs ADDRESSED	RELEVANCE TO PROJECT
1	PSO 1 — Advanced technologies for efficient design development	Transfer learning on EfficientNetB0 with a two-stage fine-tuning schedule, staged JPEG-noise augmentation and class-weighted training were used to obtain a usable detector from a modest dataset on ordinary hardware.
2	PSO 2 — Computing principles for sustainable real-world solutions	The platform answers a live societal problem — verifying the authenticity of circulating media — through a layered architecture that runs offline on commodity machines and can be extended with new forensic signals without redesign.

LIST OF PROGRAM SPECIFIC OUTCOMES (PSOs):

PSO 1: Apply advanced technologies through innovations to enhance the efficiency of design development.

PSO 2: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

MAPPING WITH SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG NO	SDGs ADDRESSED	RELEVANCE TO PROJECT
SDG 9	Industry, Innovation and Infrastructure	Builds resilient digital infrastructure for media verification and fosters innovation by making a multi-signal forensic pipeline available as open, self-hosted software.
SDG 16	Peace, Justice and Strong Institutions	Counters synthetic disinformation that erodes public trust, and supplies a documented, downloadable forensic report that can support fair inquiry rather than rumors.
SDG 4	Quality Education	The explanation panel teaches users why a verdict was reached, building practical media literacy instead of asking them to accept an opaque score.

SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG 9 — Industry, Innovation and Infrastructure: Build resilient infrastructure, promote inclusive and sustainable industrialization, and foster innovation.

SDG 16 — Peace, Justice and Strong Institutions: Promote peaceful and inclusive societies, provide access to justice for all, and build effective, accountable and inclusive institutions.

SDG 4 — Quality Education: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

DEPARTMENT OF COMPUTER APPLICATIONS

OCTOBER 2026



DECLARATION

*I hereby certify that the work which is being presented in the project entitled "**AI-POWERED DEEPFAKE AND SYNTHETIC MEDIA DETECTION SYSTEM**" submitted in the **DEPARTMENT OF COMPUTER APPLICATIONS** is an authentic record of my own work carried under the supervision of **Ms. ASHWANI VIJAYACHANDRAN,** Assistant Professor, Department of Computer Applications. This study has not been submitted to any other institution or university for the award of any other degree. This report has been checked for plagiarism by the college and the similarity index is within permissible limits set by the college.*

Date: *Name & Signature of Student*

Place: Ettumanoor **ALAN BENNY**

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

DEPARTMENT OF COMPUTER APPLICATIONS

OCTOBER 2026



CERTIFICATE

*Certified that the report entitled "**AI-POWERED DEEPFAKE AND SYNTHETIC MEDIA DETECTION SYSTEM**" submitted by "**ALAN BENNY**, Reg. No. **MLM25MCA-2014**" to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the **DEGREE OF MASTER OF COMPUTER APPLICATIONS** by **APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY** is a bonafide record of the MINI PROJECT work carried out by him under our guidance and supervision during the academic year 2025-27.*

PROJECT GUIDE
Ms. Ashwani Vijayachandran
Assistant Professor
Department of Computer Applications

PROJECT COORDINATOR
Ms. Athira R Kurup
Assistant Professor
Department of Computer Applications

HEAD OF THE DEPARTMENT
Ms. Divya S B
Associate Professor
Department of Computer Applications

INTERNAL EXAMINER

ACKNOWLEDGEMENT

I am greatly indebted to the authorities of Mangalam College of Engineering for providing the necessary facilities to successfully complete my Project on the topic "**AI-POWERED DEEPFAKE AND SYNTHETIC MEDIA DETECTION SYSTEM**".

I express my sincere thanks to **Dr. Vinodh P Vijayan**, Principal, Mangalam College of Engineering for providing the facilities to complete my Project successfully.

I thank and express my solicit gratitude to **Ms. Divya S B**, HOD, Department of Computer Applications, Mangalam College of Engineering, for her invaluable help and support which helped me a lot in successfully completing this Project work.

I express my gratitude to my Internal Guide, **Ms. Ashwani Vijayachandran**, Assistant Professor, Department of Computer Applications for the suggestions and encouragement which helped in the successful completion of my Project.

Furthermore, I would like to acknowledge with much appreciation the crucial role of the faculties especially Project coordinator, **Ms. Athira R Kurup**, Department of Computer Applications, Mangalam College of Engineering, who gave the permission to use all the required equipment and the necessary resources to complete the presentation and report.

Finally, I would like to express my heartfelt thanks to my parents who were very supportive both financially and mentally and for their encouragement to achieve my goal.

ALAN BENNY
(MLM25MCA-2014)

ABSTRACT

Synthetic media produced by generative adversarial networks and diffusion models has moved, within a very short span of years, from a research curiosity to a commodity that anyone with a browser can create. Ordinary viewers have no reliable way to tell a manipulated face apart from a genuine one, and the few detection tools that exist are either research prototypes with no usable interface or commercial services that demand a subscription and the surrender of the media being examined.

This mini project addresses that gap by building a complete, self-hosted web platform that lets any registered user submit an image, a video file, a YouTube link or an Instagram Reel and receive a reasoned verdict on its authenticity. For video input, frames are sampled across the whole timeline, each frame is scored independently, and the per-frame scores are fused using a weighted mean blended with a median so that a single anomalous frame cannot dominate the outcome.

These heterogeneous signals are combined by an ensemble decision engine whose weights and thresholds are declared in one auditable location. Every verdict is accompanied by the individual signal scores, a confidence percentage bounded to an honest range, and a downloadable forensic report in PDF form.

The platform was exercised on one hundred and ten live submissions spanning uploaded images, uploaded videos, YouTube videos and Instagram Reels. The resulting system demonstrates that transparent, explainable and honestly calibrated deepfake detection can be delivered through an ordinary web browser without specialized hardware, and that admitting uncertainty makes such a tool more trustworthy rather than less.

Keywords:

Deepfake detection, EfficientNetB0, transfer learning, media forensics, error level analysis, content provenance, ensemble decision fusion, explainable artificial intelligence, Django, digital media integrity.

Mapping with Sustainable Development Goals	Goals Addressed
SDGs to which this project contributes	SDG 9 — Industry, Innovation and Infrastructure SDG 16 — Peace, Justice and Strong Institutions SDG 4 — Quality Education

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	**i**
	ABSTRACT	**ii**
	TABLE OF CONTENTS	**iii**
	LIST OF TABLES	**v**
	LIST OF FIGURES	**vi**
	LIST OF ABBREVIATIONS	**vii**
Chapter 1	**INTRODUCTION**	**1**
1.1	Background	1
1.2	Motivation	2
1.3	Scope and Objectives	3
1.4	Problem Statement	5
1.5	Organization of the Report	5
Chapter 2	**REQUIREMENTS AND ANALYSIS**	**7**
2.1	Existing System and Problem Statement	7
2.2	Solution Overview	8
2.3	Feasibility Study	9
2.4	Requirement Specification	10
Chapter 3	**LITERATURE REVIEW**	**13**
3.1	Paper 1 — FaceForensics++	13
3.2	Paper 2 — MesoNet	13
3.3	Paper 3 — Xception and Transfer Learning	14
3.4	Paper 4 — Celeb-DF Benchmark	14
3.5	Paper 5 — EfficientNet Scaling	14
3.6	Paper 6 — Capsule-Forensics	15
3.7	Paper 7 — Frequency-Domain Artefacts	15
3.8	Paper 8 — Recurrent Temporal Detection	15
3.9	Paper 9 — DeepFake Detection Challenge	16
3.10	Paper 10 — Content Provenance and C2PA	16
3.11	Summary of the Literature	16
Chapter 4	**DIAGRAMS**	**18**
4.1	Data Flow Diagrams	18
4.2	Use Case Diagram	20
4.3	Class Diagram	20

CHAPTER NO.	TITLE	PAGE NO.
4.4	State Diagram	21
4.5	Activity Diagram	22
4.6	Sequence Diagram	23
4.7	Entity Relationship Diagram	23
Chapter 5	**METHODOLOGY**	**25**
5.1	Module Descriptions	25
5.2	Algorithms	32
5.3	Architecture	35
5.4	Case Study	36
Chapter 6	**IMPLEMENTATION DETAILS**	**38**
6.1	Implementation Procedures	38
6.2	Testing Methods and Results	41
Chapter 7	**ADVANTAGES AND DISADVANTAGES**	**45**
7.1	Advantages	45
7.2	Disadvantages	46
Chapter 8	**RESULT AND DISCUSSION**	**48**
8.1	Analysis	48
Chapter 9	**CONCLUSION**	**53**
9.1	Summary	53
9.2	Limitations	53
9.3	Future Scope	54
9.4	Applications	55
Chapter 10	**APPENDICES**	**56**
10.1	Appendix 1 — Sample Code of the Ensemble Decision Engine	56
10.2	Appendix 2 — Sample Code of the Video Analysis Module	58
Chapter 11	**REFERENCES**	**61**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparison of existing deepfake verification options	8
2.2	Minimum and recommended hardware requirements	10
2.3	Software requirements and versions	10
2.4	Functional requirement summary	11
3.1	Consolidated summary of the reviewed literature	16
5.1	Summary of implemented modules and responsibilities	25
5.2	Forensic signal weights used in the analysis module	28
5.3	Ensemble weights for active-CNN and heuristic modes	33
5.4	Classification tiers and score thresholds	33
6.1	Application database tables and key columns	39
6.2	Dataset composition and split	42
6.3	Unit and integration test suite results	42
6.4	Representative system test cases	43
8.1	Distribution of verdicts across 110 live detections	48
8.2	Mean processing latency by input channel	50
8.3	Operating mode of the intelligence layer	51
8.4	Achievement against the objectives stated in Chapter 1	52

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Data flow diagram — Level 0 (context diagram)	18
4.2	Data flow diagram — Level 1	18
4.3	Data flow diagram — Level 2 (analysis subsystem)	19
4.4	Use case diagram of the detection platform	20
4.5	Class diagram of domain and service classes	21
4.6	State diagram of a submission lifecycle	21
4.7	Activity diagram of the detection workflow	22
4.8	Sequence diagram of a single detection request	23
4.9	Entity relationship diagram of the database schema	24
5.1	Registration and login interface	26
5.2	Media upload interface with channel selection	27
5.3	Detection result page with signal breakdown	28
5.4	Analytics dashboard with distribution and trend charts	29
5.5	Detection history listing	30
5.6	Generated PDF forensic report	30
5.7	Landing page of the detection platform	31
5.8	User profile page	32
5.9	System architecture of the detection platform	36
8.1	Distribution of classification tiers over 110 detections	48
8.2	Composition of analyzed media by input channel	59
8.3	Confidence distribution across recorded detections	50
8.4	Observed latency by input channel	51

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Receiver Operating Characteristic Curve
C2PA	Coalition for Content Provenance and Authenticity
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSRF	Cross-Site Request Forgery
DCT	Discrete Cosine Transform
DFD	Data Flow Diagram
DNN	Deep Neural Network
ELA	Error Level Analysis
EXIF	Exchangeable Image File Format
FPS	Frames Per Second
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
HSV	Hue, Saturation, Value color space
HTTP	Hypertext Transfer Protocol
IPTC	International Press Telecommunications Council metadata
JPEG	Joint Photographic Experts Group
JUMBF	JPEG Universal Metadata Box Format
LR	Learning Rate
MCA	Master of Computer Applications
ML	Machine Learning
MVT	Model-View-Template
ORM	Object Relational Mapper
PDF	Portable Document Format
PNG	Portable Network Graphics
RGB	Red, Green, Blue color space
ROC	Receiver Operating Characteristic
SDG	Sustainable Development Goal
SQL	Structured Query Language
SSRF	Server-Side Request Forgery
UI	User Interface

ABBREVIATION	EXPANSION
UML	Unified Modeling Language
URL	Uniform Resource Locator
WSGI	Web Server Gateway Interface
XMP	Extensible Metadata Platform

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

A deepfake is a photograph, an audio clip or a video sequence in which a person appears to say or do something that never happened, produced by a machine learning model rather than by conventional editing. Neither approach now requires a research laboratory: mobile applications and free web services perform the same synthesis in seconds. Because such content spreads through messaging applications and social platforms, a single convincing clip can reach very large audiences long before anyone has had the opportunity to question where it came from. The lowering of the technical barrier is therefore not merely a matter of convenience for creators; it changes the economics of deception, since producing a believable falsehood now costs far less effort than debunking it.

The technical reason this matter is that synthetic media no longer carries the obvious tells that editing once left behind. The artefacts that remain are subtle and frequently invisible to the eye: faint periodic patterns left by transposed convolution layers, unnaturally uniform local noise, colour saturation that falls outside the range a real camera sensor produces, and micro-inconsistencies in how a face moves between successive frames. For this reason the traditional habit of looking for visible glitches, mismatched lighting or awkward blinking is no longer a dependable defence. Each new generation of models removes the weaknesses that the previous generation exposed, and a detector or a human observer calibrated to yesterday's flaws will tend to miss tomorrow's. Reliable analysis consequently has to look below the level of what the eye perceives and measure statistical properties of the file itself.

Alongside the technical shift there has been a social one. The practical consequence is that verification must be available to the ordinary viewer at the moment of doubt, not only to institutions after the fact. The social dimension also includes the erosion of trust in genuine material: once viewers know that any recording might be fabricated, authentic evidence can be dismissed as fake as readily as fake evidence is accepted as real. A usable verification tool therefore serves two purposes, helping people recognise manipulated media and helping them defend the credibility of material that is genuine.

Research into automated detection has advanced in parallel with generation. A separate industry effort has produced content provenance standards that embed a signed record of how a piece of media was created directly into the file. These two lines of work are complementary rather than competing. Detection attempts to infer manipulation from the content after the fact, whereas provenance attempts to establish origin at the moment of capture or creation, and each is most informative precisely where

the other is silent. Neither, however, has so far been packaged in a form that a non-specialist can apply to an arbitrary file or link.

Each of these approaches has a characteristic weakness. This observation — that no single signal is dependable but the signals fail in different circumstances — is the technical premise on which the present project is built. In practice this means that a classifier may be misled by recompression, that frequency-based forensics may be blunted by resizing, and that provenance metadata disappears as soon as a platform strips the file container. Designing a system around the combination of such signals, together with an honest account of when they disagree, is the approach followed throughout the remainder of this report.

1.2 MOTIVATION

The motivation for this work came from observing the gap between what detection research has achieved and what an ordinary person can actually use. Or they can find a research repository, install a deep learning environment, obtain model weights, and run a command-line script — a path that excludes almost everyone. In the course of reviewing available tools it became clear that the barrier is rarely the quality of the underlying algorithm; it is the absence of an interface, a deployment path and an explanation that a student, a journalist or a concerned relative could actually use. Closing that usability gap, rather than improving a benchmark figure by a fraction of a percentage point, was treated as the primary contribution of the project.

A second motivation concerns the manner in which detection results are presented. The literature is consistent that detector accuracy collapses when the generation method is unfamiliar, which means that overconfident output is not an occasional flaw but an expected one. Presenting a single percentage without context invites the reader to treat it as a fact. The project therefore reports the contributing signals alongside the verdict, bounds the confidence it is willing to claim and attaches a written caution, so that the person reading the result understands both what the system concluded and how much weight that conclusion can reasonably bear.

The third motivation was the observation that most detectors refuse to say "I do not know". Treating uncertainty as a first-class outcome rather than a failure is central to the design adopted here. A detector that must always choose between real and fake will inevitably label some ambiguous media incorrectly, and the cost of such an error falls on the person who relied on it. Allowing the system to report that the evidence is insufficient or contradictory returns the final judgement to the human reader in exactly those cases where a machine has least justification for deciding.

Finally, there was a pedagogical motivation. Every verdict is accompanied by an explanation of which signals contributed and why, with the intention that repeated use of the tool builds the user's own

understanding of how synthetic media can be recognised, rather than training the user to defer blindly to an automated score.

1.3 SCOPE AND OBJECTIVES

1.3.1 Scope

The project delivers a self-hosted web application that accepts media through four channels and returns an explained authenticity verdict together with a downloadable forensic report. The emphasis is on a complete, usable path from submission to documented result, so that the system can be demonstrated, tested and evaluated as a whole and not merely as a collection of isolated analysis routines.

- Four input channels are supported: direct image upload, direct video upload, analysis of a public YouTube video by URL, and analysis of a public Instagram Reel by URL. Each channel converges on the same detection service after the media has been retrieved and validated, which keeps the analysis behaviour identical regardless of how the file arrived.
- Analysis is focused on facial manipulation and whole-image synthesis in the visual domain.
- Detection combines three independent evidence sources — a convolutional neural network, a forensic signal analyzer and a provenance metadata reader — fused by a configurable decision engine. Because the three sources observe different properties of the media, namely learned appearance, low-level statistical residues and recorded creation history, a weakness in one is not automatically a weakness in the others.
- Every user has an authenticated account with a profile, a private detection history and personal analytics; results of one user are not visible to another. Access control is enforced on the server for every protected route, so privacy does not depend on the interface merely hiding information.
- Each detection can be exported as a PDF forensic report containing the verdict, the individual signal scores and a written explanation. The report is intended to be a durable record that can be filed, shared or attached to a complaint, and it carries a closing disclaimer stating that the analysis is probabilistic.
- The system is designed to run on a single commodity machine without a dedicated graphics processor, using only free and open-source components and a file-based database. This constraint was adopted deliberately so that the platform could be adopted by a small department or an individual without any procurement or cloud expenditure.

Explicitly outside scope are real-time detection of live video streams, browser extensions, mobile applications, distributed or clustered deployment, and any claim of legal admissibility for the reports

produced. These exclusions are stated at the outset so that the evaluation in later chapters is judged against what the project actually set out to deliver and not against capabilities it never claimed.

1.3.2 Objectives

The project set out to meet the following specific objectives.

- To design and train a convolutional classifier capable of distinguishing authentic facial imagery from generated or manipulated imagery, using transfer learning so that acceptable performance is reachable from a modest dataset. Transfer learning was preferred over training from scratch because the available dataset is small, and a pretrained backbone brings general visual features that would otherwise require a far larger corpus to learn.
- To implement independent forensic analysis of error level residues, frequency-domain irregularity, local noise uniformity, color saturation profile and sharpness signature, so that a verdict does not rest on the neural model alone. The five signals were selected because each responds to a different physical or statistical property of an image and is therefore unlikely to fail for the same reason as the others.
- To implement provenance inspection that recognizes generation fingerprints left in EXIF, XMP, PNG text chunks and C2PA content-credential blocks. Provenance evidence, where present, is among the strongest available, because it is a direct statement by the generating tool rather than an inference from pixels.
- To extend single-image analysis to video through intelligent frame sampling, per-frame scoring and temporal aggregation that is robust to individual outlier frames. Per-frame scores are combined so that a single corrupted or unusual frame cannot decide the outcome of a whole video.
- To construct an ensemble decision engine whose weights and thresholds are declared in one auditable place and which emits an explicit uncertain verdict when evidence is insufficient or contradictory. Keeping every weight and threshold in a single module makes the decision rules open to review and adjustment without searching through the application code.
- To generate a plain-language explanation for every verdict describing which signals contributed and what reservations apply. The explanation is composed from the actual values produced during analysis and is not a fixed template.
- To deliver the whole capability through a responsive web interface with authentication, per-user history, analytics and PDF export. The interface is designed to be understandable on first use, with the progress of an analysis and its eventual outcome presented in plain terms.

- To validate the system through automated unit and integration tests and through a body of live submissions across all four input channels. The testing objective covers both automated checks of individual components and end-to-end trials with real submissions, so that functional correctness and practical behaviour are examined together.

1.4 PROBLEM STATEMENT

Synthetic media can now be produced faster and more cheaply than it can be verified, and the people most exposed to it have no practical means of checking what they are shown. The consequence is an imbalance of effort between those who fabricate and those who must verify, and the imbalance widens as generation tools improve and become easier to operate.

The problem is compounded by the fact that individual detection signals are each defeatable in isolation. A system that relies on any one of these will be confidently wrong in a predictable set of situations. Examples include recompression that erases high-frequency traces, resizing that disturbs spectral patterns, metadata that is stripped on upload, and generators that were not represented in a classifier's training data. Each is a realistic, everyday event rather than a contrived attack.

What is required, therefore, is a system that gathers several independent forms of evidence about the same media, combines them under transparent and adjustable rules, reports not only its conclusion but the reasoning and the individual signals behind it, and declines to give a definitive answer when the evidence does not support one. This mini project implements such a system. The remaining chapters document how this requirement was analysed, designed, built and evaluated, and how closely the finished system meets it.

1.5 ORGANIZATION OF THE REPORT

The remainder of this report is organized as follows. The chapters follow the natural order of the work: understanding the problem, studying prior research, modelling the system, building it and finally evaluating it.

Chapter 3 reviews ten publications covering benchmark datasets, convolutional architectures applied to detection, frequency-domain and temporal forensics, and the content provenance approach, and states for each the specific design decision it influenced. This review is deliberately selective, favouring works whose findings translated into concrete decisions over works that are merely adjacent to the topic.

Chapter 5 describes the methodology, covering each implemented module in turn, the three principal algorithms expressed as pseudocode, the four-layer architecture and a worked case study tracing one submission from upload to report. The case study is included because it shows how the separate

modules interact on a real submission, which is difficult to convey from the module descriptions alone.

Chapter 7 states the advantages and disadvantages of the delivered system without softening either. Chapter 10 contains the appendices with sample code from the two principal modules, and Chapter 11 lists the references. The appendices reproduce source excerpts so that the central decision logic can be inspected directly alongside the narrative description.

CHAPTER 2

REQUIREMENTS AND ANALYSIS

2.1 EXISTING SYSTEM AND PROBLEM STATEMENT

2.1.1 How verification is done today

In the absence of an accessible tool, the dominant method of verifying a suspicious video remains visual inspection by the viewer. Controlled studies consistently report that human accuracy on modern synthetic faces sits close to chance, and, more troublingly, that viewer confidence is unrelated to viewer correctness. This reliance on perception is particularly weak for video, where motion and compression artefacts further mask the small inconsistencies that distinguish synthetic content, and where the viewer's attention is naturally directed at the message rather than at the pixels.
Where automated checking is used, it generally takes one of three forms.

2.1.2 Limitations of the existing options

The following limitations were identified during analysis and directly shaped the requirements for the proposed system. Each limitation is revisited in the solution overview, where the corresponding design response is identified.

- **Cost and access barrier.** Commercial verification is typically priced per query or by subscription, which places routine use beyond the reach of students, small newsrooms and ordinary individuals.
- **Loss of custody over the media.** Uploading a private or sensitive clip to a third-party service means surrendering control of it, which is unacceptable in many personal and institutional settings.
- **Single-signal fragility.** A tool built on one classifier fails in a predictable way whenever the input falls outside what that classifier was trained to recognise.
- **Opacity of output.** A bare score or label gives the reader nothing to evaluate, challenge or learn from.
- **Absence of an uncertain outcome.** Forcing every input into real or fake guarantees that ambiguous cases will be mislabelled with apparent authority.
- **Narrow input support.** Many tools accept only a local image or a short clip and cannot examine media that exists only as a link on a social platform.
- **No durable record.** Without a stored result or exportable report, a judgement cannot be revisited, cited or compared with later analysis.

- **Specialist deployment requirements.** Research code usually demands a configured deep learning environment, compatible libraries and often a graphics processor.

Table 2.1: Comparison of existing deepfake verification options

Criterion	Manual visual inspection	Commercial online service	Research reference code	Proposed system
Cost to the user	None	Subscription	None, but requires expertise	None
Media stays local	Yes	No	Yes	Yes
Reliability on modern fakes	Near chance	Moderate, undisclosed	Varies by dataset	Moderate with stated limits
Number of independent signals	One (human)	Usually one	Usually one	Three, fused
Explanation of the verdict	Subjective	None or minimal	Raw scores only	Written explanation plus per-signal scores
Uncertain verdict possible	Informally	Rarely	No	Yes, explicit tier
Social media link input	Not applicable	Sometimes	No	YouTube and Instagram
History and analytics	No	Limited	No	Per-user history and dashboard
Exportable report	No	Sometimes	No	PDF forensic report
Hardware requirement	None	None (remote)	Often a GPU	Commodity CPU machine

2.2 SOLUTION OVERVIEW

This is a Django 5 web app, self-hostable on standard hardware.

Its core design splits analysis into three independent producers feeding one fusing consumer. For video, all three run per-frame (uniformly sampled, capped at 60), weighting face-containing frames double, and fusing via weighted mean + median to resist outliers, with standard deviation yielding a temporal-consistency signal. This separation keeps the producers independent, so that each can be developed, tested and replaced without changing the others, and makes the contribution of every signal to a final verdict traceable.

The decision engine normalizes all inputs to 0–1, weights them by model availability, and fuses a score — applying guardrails that force "uncertain" verdicts on signal disagreement, missing faces in fallback mode, or ambiguous mid-range scores, while letting strong network signals override otherwise-mild fusion. Because the guardrails can only move a verdict towards uncertainty, the

engine errs on the side of declining to commit rather than asserting a conclusion the evidence does not support.

The app also includes account management, private detection history, an analytics dashboard, and a report archive. These supporting features turn the detector from a one-off utility into a usable service in which results accumulate, can be reviewed over time and can be exported when required.

2.3 FEASIBILITY STUDY

2.3.1 Technical feasibility

All components rely on mature open-source tools: Django 5 (web/auth/ORM), TensorFlow/Keras with pretrained EfficientNetB0 (enabling transfer learning), OpenCV (video, face detection, DCT/Laplacian), Pillow (ELA, metadata), ReportLab (PDFs), and yt-dlp/instaloader (media retrieval). Because every dependency is open source and widely documented, the project can be reproduced on another machine from the setup procedure alone, and the risk of abandonment or licensing change is low.

Two risks were mitigated: training a usable classifier on limited hardware/data (solved via two-stage transfer learning — frozen backbone first, then unfreezing the top 30 layers at low learning rate, plus augmentation and simulated JPEG noise), and non-GPU inference latency (solved by capping frame counts and downsizing forensic computations). Measured results confirmed both fixes worked. These mitigations show that technical feasibility was established by experiment on the target hardware and not merely assumed.

2.3.2 Economic feasibility

The direct monetary cost of the project is zero. By contrast, per-query commercial verification would accumulate cost indefinitely. Hardware requirements are modest enough that an existing departmental computer can host the system, so no capital expenditure is needed either. The time invested by the developer is the only substantial resource consumed, and it is bounded by the semester schedule.

2.3.3 Operational feasibility

The interface follows conventions already familiar from any file-sharing website: register, log in, choose a file or paste a link, wait, read the result. The system therefore fits the operational capability of a college department or a small organisation without a dedicated specialist. Registration, history and reports follow familiar patterns, so little or no training is needed, and error messages are written in plain language to guide the user when a file is rejected.

2.3.4 Legal and ethical feasibility

Three ethical concerns were identified and addressed in design. Third, the system itself must not become an attack vector: submitted URLs are validated against a block list of loopback and private address ranges to prevent server-side request forgery, only HTTP and HTTPS schemes are accepted, and upload size and extension are validated before any file is written. Alongside this technical safeguard, the system limits harm from misinterpretation by bounding its confidence, using an explicit uncertain tier, isolating each user's results and attaching a disclaimer to every report.

2.3.5 Schedule feasibility

The work was decomposed into six increments — environment and dataset preparation, model training and evaluation, the Django core with authentication, the detection pipeline and ensemble engine, the social-link channels and PDF reporting, and finally testing and documentation. Each increment produced a working, testable state of the system, so that progress could be reviewed with the guide at regular intervals and defects were found while the relevant code was still fresh. The sequence placed the riskiest work, model training, before the web layer so that any difficulty there would surface early.

2.4 REQUIREMENT SPECIFICATION

2.4.1 Hardware requirements

Table 2.2: Minimum and recommended hardware requirements

Component	Minimum	Recommended
Processor	Dual-core x86-64, 2.0 GHz	Quad-core x86-64, 2.5 GHz or higher
System memory	4 GB RAM	8 GB RAM or more
Free disk space	5 GB (application, weights, database)	20 GB including dataset and media store
Graphics processor	Not required	CUDA-capable GPU for faster training
Display	1366 × 768	1920 × 1080
Network	Required only for URL-based analysis	Broadband for video retrieval

2.4.2 Software requirements

Table 2.3: Software requirements and versions

Category	Component	Version
Operating system	Windows 11	25H2
Language runtime	Python	3.10 or 3.11

Category	Component	Version
Web framework	Django	5.0 or later
Deep learning	TensorFlow / Keras	3.0 or later
Computer vision	OpenCV-Python	4.8 or later
Numerical computing	NumPy, SciPy, scikit-learn	Current stable
Imaging	Pillow	10.0 or later
Report generation	ReportLab	4.0 or later
Media retrieval	yt-dlp, instaloader	Current stable
Front-end	Bootstrap, Chart.js	5.x, 4.x
Database	SQLite	3.x (bundled with Python)
Version control	Git	Any current release

2.4.3 Functional requirements

Table 2.4: Functional requirement summary

ID	Requirement	Priority
FR-01	The system shall allow a visitor to register an account and an existing user to log in and log out securely.	High
FR-02	The system shall allow an authenticated user to upload an image in JPG, PNG, WebP, BMP or TIFF format up to 10 MB.	High
FR-03	The system shall allow an authenticated user to upload a video in MP4, AVI, MOV, MKV, WebM or similar format up to 100 MB.	High
FR-04	The system shall accept a public YouTube URL, retrieve the video and analyse it.	High
FR-05	The system shall accept a public Instagram Reel URL, retrieve the media and analyse it.	Medium
FR-06	The system shall analyse submitted media using a convolutional classifier, a forensic analyser and a provenance reader.	High
FR-07	The system shall fuse the available signals and assign one of five classification tiers.	High
FR-08	The system shall return an explicit uncertain verdict when signals contradict, no face is found in fallback mode, or the fused score is inconclusive.	High
FR-09	The system shall display the individual signal scores and a written explanation alongside every verdict.	High
FR-10	The system shall generate and serve a PDF forensic report for any detection.	Medium
FR-11	The system shall maintain a private detection history for each user.	Medium
FR-12	The system shall present an analytics dashboard with verdict distribution and a seven-day activity trend.	Medium
FR-13	The system shall allow a user to edit a profile, upload a profile picture and change a password.	Low

ID	Requirement	Priority
FR-14	The system shall archive every generated report and allow a user to retrieve a previously generated report without regenerating it.	Low

2.4.4 Non-functional requirements

- **Performance.** A single image should be analyzed within approximately fifteen seconds and a short video within approximately three minutes on the minimum specified hardware.
- **Reliability.** A failure in any individual signal must degrade the verdict towards uncertainty rather than crash the request, and no random or placeholder value may ever be substituted for a real score.
- **Security.** All detection views require authentication, form submissions carry cross-site request forgery protection, uploads are validated for type and size, and supplied URLs are checked against private address ranges.
- **Usability.** The interface must be operable by a non-specialist, responsive across desktop and mobile widths, and must state the meaning of every number it displays.
- **Maintainability.** Detection logic must reside outside the view layer, and all fusion weights and thresholds must be declared in a single module so that tuning does not require code archaeology.
- **Portability.** The application must run on Windows, Linux and macOS with no code change, using a file-based database and relative paths.
- **Privacy.** Media submitted by one user must not be visible to another, and media fetched from external platforms must be deleted from temporary storage once analysis completes**.**

CHAPTER 3

LITERATURE REVIEW

This chapter reviews ten publications that informed the design of the present system. Each review notes the contribution of the work and the specific way it influenced decisions taken in this project. The papers were chosen to span the three evidence families used in the system, namely learned appearance, signal-level forensics and provenance, together with the datasets and benchmarks against which such methods are judged.

3.1 PAPER 1 — FACEFORENSICS++: LEARNING TO DETECT MANIPULATED FACIAL IMAGES

Rössler and colleagues assembled a large-scale corpus of facial manipulations built from a thousand source videos processed through four distinct manipulation techniques covering both computer-graphics-based and learning-based approaches. Crucially, each video was released at three compression levels, reflecting the reality that media circulates after heavy re-encoding rather than in pristine form. The authors benchmarked both hand-crafted forensic features and learned classifiers across these conditions and reported a consistent finding: detection accuracy that appears strong on raw video degrades substantially as compression increases, and hand-crafted features degrade faster than learned ones. The importance of this work for the present project lies less in any single result than in its demonstration that evaluation conditions matter: a detector assessed only on pristine video says little about how it will behave on a clip downloaded from a social platform.

Two conclusions from this work were adopted directly. Second, because compression damages the very high-frequency residues that classical forensics depends on, forensic signals should be treated as supporting evidence with a modest weight rather than as the primary determinant — a decision reflected in the weight profile of the ensemble engine. Together these conclusions explain why the training procedure includes simulated JPEG noise and why the network dominates the fused score whenever it is operating normally.

3.2 PAPER 2 — MESONET: A COMPACT FACIAL VIDEO FORGERY DETECTION NETWORK

Afchar and colleagues argued that detection should operate at a mesoscopic level of image description, between the microscopic level of pixel noise, which compression destroys, and the macroscopic level of semantic content, where synthetic faces are already convincing. The resulting networks were small, with only a few layers and a modest number of parameters, which made them fast to train and easy to analyse. The idea of looking at intermediate-scale structure rather than at

either extreme is echoed in the choice here of a pretrained backbone whose mid-level features capture exactly that kind of texture and part structure.

The relevance to this project is twofold.

3.3 PAPER 3 — XCEPTION AND TRANSFER LEARNING FOR MANIPULATION DETECTION

A body of work established that convolutional architectures pre-trained on general image classification transfer effectively to manipulation detection when the classification head is replaced and the network fine-tuned. Pretrained networks have already learned edges, textures and shapes from millions of natural images, and these low-level and mid-level features remain useful when the task changes from object recognition to distinguishing authentic from manipulated faces. Only the final decision layers need to be learned from the comparatively small manipulation dataset.

This finding underpins the training strategy used here. This two-stage schedule is a direct application of the transfer learning practice established in this literature. Training first with the backbone frozen allows the new classification head to settle without disturbing the pretrained features, and the subsequent low-rate fine-tuning of the top layers adapts the representation to the manipulation task with limited risk of overfitting a small corpus.

3.4 PAPER 4 — CELEB-DF: A LARGE-SCALE CHALLENGING DATASET

Li and colleagues observed that detectors reporting near-perfect accuracy on early datasets were in part exploiting visible artefacts — colour mismatch at blending boundaries, low-resolution synthesised regions, temporal flicker — that arose from the synthesis pipelines of the time rather than from any fundamental property of synthetic media. The lesson is that high scores on a benchmark may reflect the quirks of the benchmark rather than a general ability to recognise manipulation. The authors responded by building a higher-quality dataset in which such shortcuts are far less available, and detector performance on it dropped markedly.

This paper is the strongest single argument in the reviewed literature for the design philosophy adopted here. The response in this project is structural: multiple independent signals, an explicit contradiction check between them, a mandatory uncertain tier, and a confidence value bounded to a range that does not claim near-certainty. In other words, humility about generalisation is encoded into the system architecture rather than left as a footnote.

3.5 PAPER 5 — EFFICIENTNET: RETHINKING MODEL SCALING

Tan and Le showed that the conventional practice of scaling a convolutional network along a single dimension — depth, width or input resolution — is suboptimal, and proposed a compound scaling

method that adjusts all three according to a fixed ratio. This produced a family of models that achieve strong accuracy with substantially fewer parameters and less computation than earlier networks of comparable performance.

EfficientNetB0, the baseline member of that family, was selected as the backbone for this project precisely because of this efficiency property. The measured image analysis latency reported in Chapter 8 confirms that inference on the resulting model is fast enough for interactive web use on commodity hardware. A smaller, efficient backbone also reduces memory consumption and keeps the trained weight files compact enough to be distributed with the project.

3.6 PAPER 6 — CAPSULE-FORENSICS FOR DETECTING FORGED IMAGES AND VIDEOS

Nguyen and colleagues applied capsule networks, which encode part-whole spatial relationships through vector-valued activations and dynamic routing, to forgery detection. The attraction of the approach is that it can represent how parts of a face relate to one another, which a plain convolutional stack may not capture explicitly, and the reported results were competitive on several forgery types.

Although a capsule architecture was not adopted here — the implementation complexity and the immaturity of tooling made it unsuitable for a single-semester project — the paper contributed an important idea. The project applies that idea across signal families rather than across neural architectures. In effect, redundancy through diversity of viewpoint is more valuable than additional depth in a single viewpoint.

3.7 PAPER 7 — FREQUENCY-DOMAIN ARTEFACTS OF GENERATIVE MODELS

Studies by Frank et al.

This motivated the forensic module's DCT component: images/frames are reduced to a fixed grid, transformed via 2D cosine transform, and scored on spectral energy outside the low-frequency band. Generative upsampling leaves periodic traces in the spectrum of an image that are often invisible in the pixel domain but distinct in the frequency domain, and the paper showed that such traces can help separate generated images from camera photographs. Because they are weakened by compression and resizing, the resulting signal is used as supporting evidence rather than as proof.

3.8 PAPER 8 — RECURRENT NETWORKS FOR TEMPORAL DEEPFAKE DETECTION

Güera and Delp observed that face-swapping pipelines generate each frame independently and therefore fail to enforce consistency across time. They proposed a pipeline combining a convolutional feature extractor with a recurrent layer over a sequence of frames, and reported that reliable classification was achievable from a short sequence of only a few dozen frames. Their work is also a

reminder that manipulation is frequently easiest to see across time, since an artefact that is almost imperceptible in one frame can appear as flicker when frames are viewed in sequence.

The video subsystem in this project is a simplified realisation of the same insight. The finding that a few dozen frames suffice also justified the cap of sixty analysed frames, which bounds latency without discarding useful evidence. The present system does not train a recurrent network, but the statistical spread of per-frame scores is retained as a temporal-consistency indicator, preserving some of the benefit of sequence analysis at much lower cost.

3.9 PAPER 9 — THE DEEPFAKE DETECTION CHALLENGE

The competition's organizers documented a major detection competition's dataset and results — thousands of entrants, winners relying on CNN ensembles with heavy augmentation. The scale of the competition made it the most realistic public test of detection under unfamiliar conditions, and the gap between validation results and results on hidden test data was one of its most widely discussed outcomes.

This shaped two choices here: ensembling across signal types (not just model instances) as a core design principle, and capped confidence reporting — since strong in-distribution results don't guarantee generalization, the engine bounds its confidence output and every report states the result is probabilistic. The competition therefore reinforced the position that detectors should be evaluated and reported with caution, and that agreement between different kinds of evidence is more persuasive than a single high score.

3.10 PAPER 10 — CONTENT PROVENANCE AND THE C2PA SPECIFICATION

The C2PA has published a spec for signed content credentials tracking media origin and edits.

This shaped the provenance module built here: it scans EXIF/XMP, PNG chunks, and JUMBF blocks against known generation-tool signatures. The module therefore treats a positive finding as informative and an absent one as inconclusive, since most media carries no credentials at all and their absence proves nothing.

3.11 SUMMARY OF THE LITERATURE

Table 3.1: Consolidated summary of the reviewed literature

No.	Work	Principal contribution	Influence on this project
1	FaceForensics++	Benchmark corpus at three compression levels; accuracy degrades with compression	Added JPEG-noise augmentation; forensic signals weighted below the network
2	MesoNet	Compact mesoscopic networks detect face swaps efficiently	Justified a small backbone and face-region cropping

No.	Work	Principal contribution	Influence on this project
3	Xception transfer learning	ImageNet pre-training transfers strongly to manipulation detection	Two-stage frozen-then-fine-tuned training schedule
4	Celeb-DF	Detector accuracy collapses on harder, unseen synthesis	Mandatory uncertain tier and bounded confidence
5	EfficientNet	Compound scaling gives accuracy at low parameter cost	Selection of EfficientNetB0 as the backbone
6	Capsule-Forensics	Different architectures capture different evidence	Rationale for fusing heterogeneous signal families
7	Frequency-domain artefacts	Generative models leave periodic spectral signatures	DCT high-frequency energy ratio in the forensic module
8	Recurrent temporal detection	Independently generated frames flicker over time	Temporal consistency measure and 60-frame sampling cap
9	DeepFake Detection Challenge	Large gap between familiar and unseen data; ensembles win	Ensemble design and constrained confidence reporting
10	C2PA content provenance	Signed credentials certify origin but are strippable	Provenance module; absence treated as weak evidence only

Taken together, the reviewed literature supports a consistent position. The architecture described in the following chapters is a direct response to that position. The literature also shows what remains unsolved: generalisation to new generators, robustness to compression and the fragility of any single cue. These open problems define the limits within which the present system operates and are acknowledged in the limitations discussed in Chapter 9.

CHAPTER 4

DIAGRAMS

This chapter presents the structural and behavioral models of the system. The unified modelling language diagrams that follow describe actor interaction, static structure, lifecycle states, control flow and message ordering respectively.

4.1 DATA FLOW DIAGRAMS

A data flow diagram models a system as a set of processes that transform data, external entities that supply or consume it, and data stores that retain it. The levelled approach shown here begins with a single-process context view and progressively decomposes the processes that carry the most complexity.

4.1.1 DFD Level 0 — Context diagram

The context diagram establishes the system boundary. Everything else is treated here as a single process.

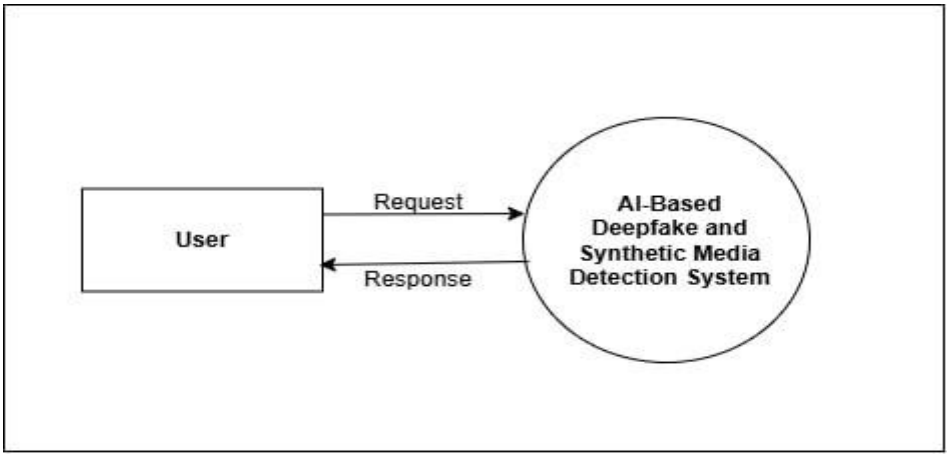


Figure 4.1: Data flow diagram — Level 0 (context diagram)

4.1.2 DFD Level 1

The first level of decomposition splits the single process into six. 0 renders a PDF forensic report on demand and records its location.

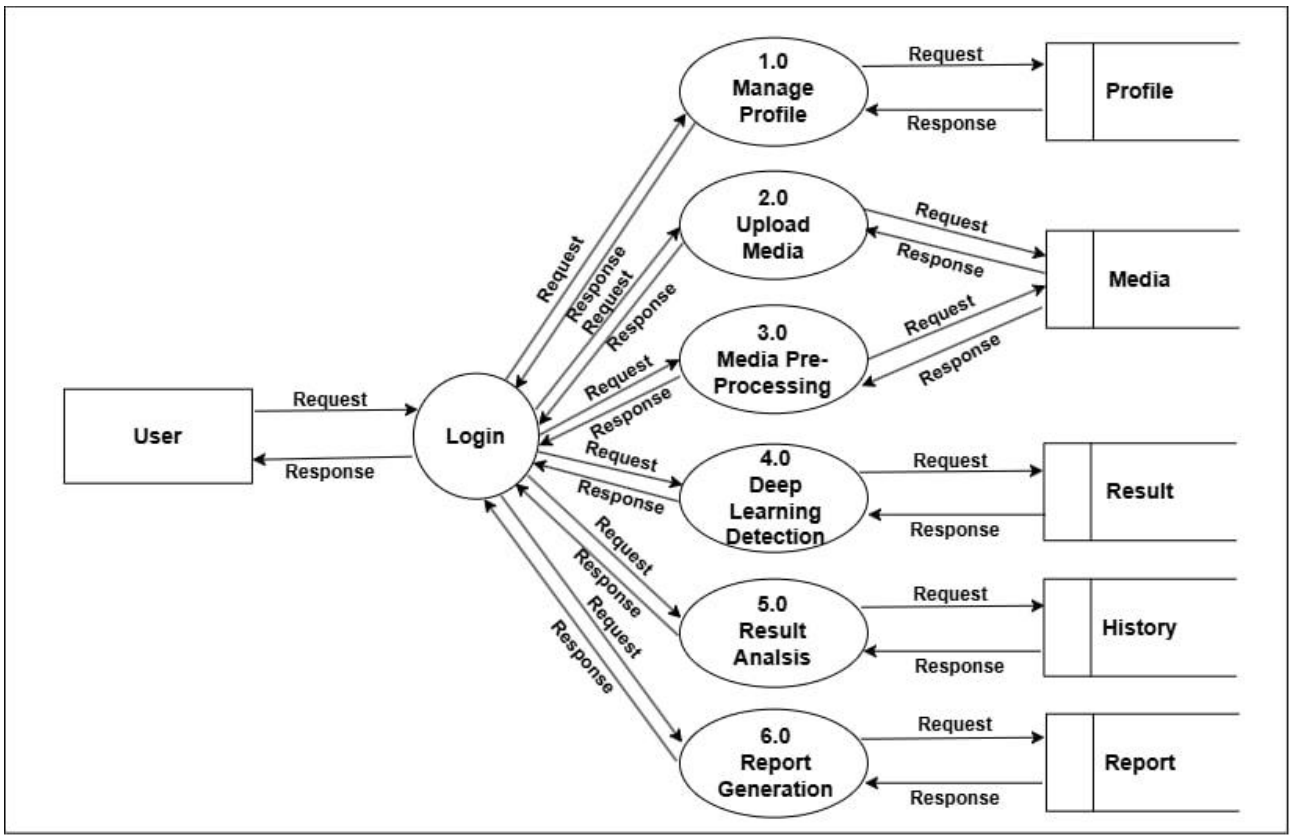


Figure 4.2: Data flow diagram — Level 1

4.1.3 DFD Level 2 — Analysis subsystem

Process 4. The four sub-processes are mutually independent and their outputs form the signal vector consumed by the ensemble engine.

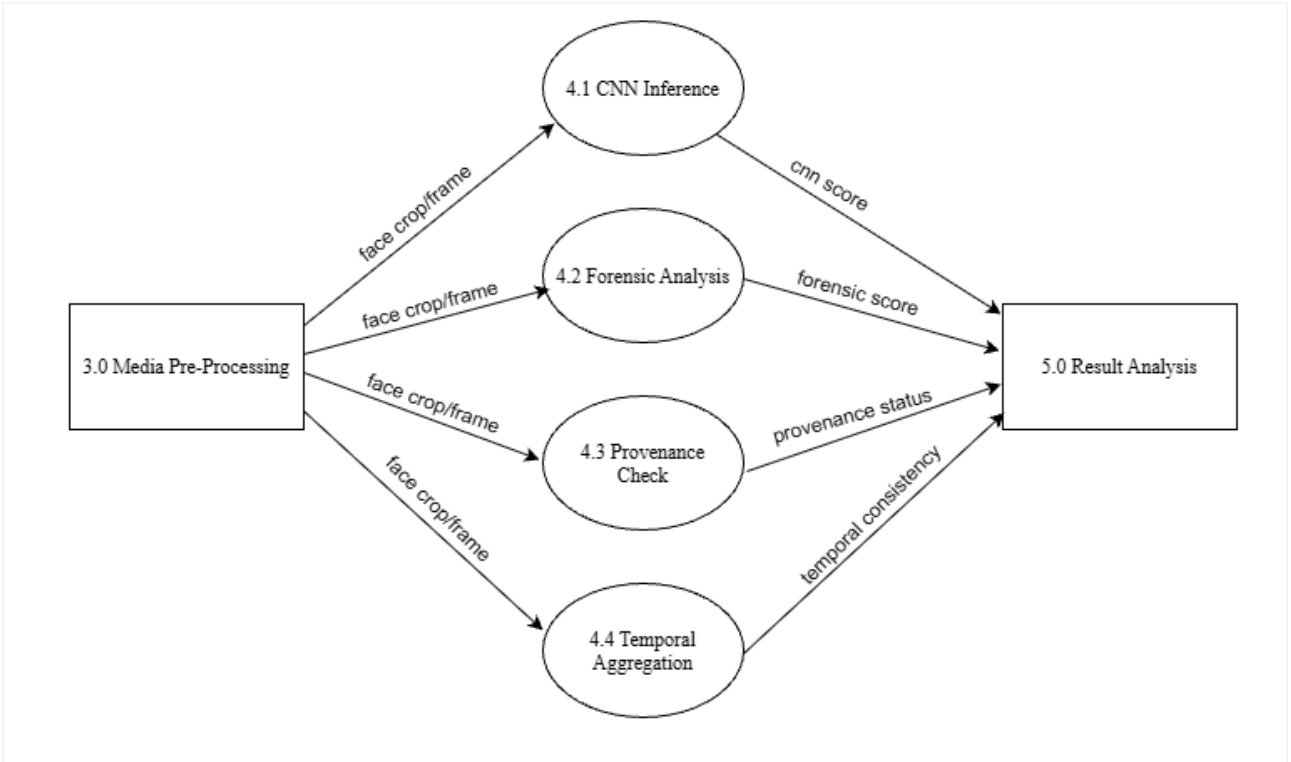


Figure 4.3: Data flow diagram — Level 2 (analysis subsystem)

4.2 USE CASE DIAGRAM

The use case diagram identifies the single actor and the services available to them. Every use case except registration, login and password reset requires an authenticated session, which is enforced by a decorator at the view layer rather than by hiding links in the template, so a direct request to a protected route is redirected rather than served.

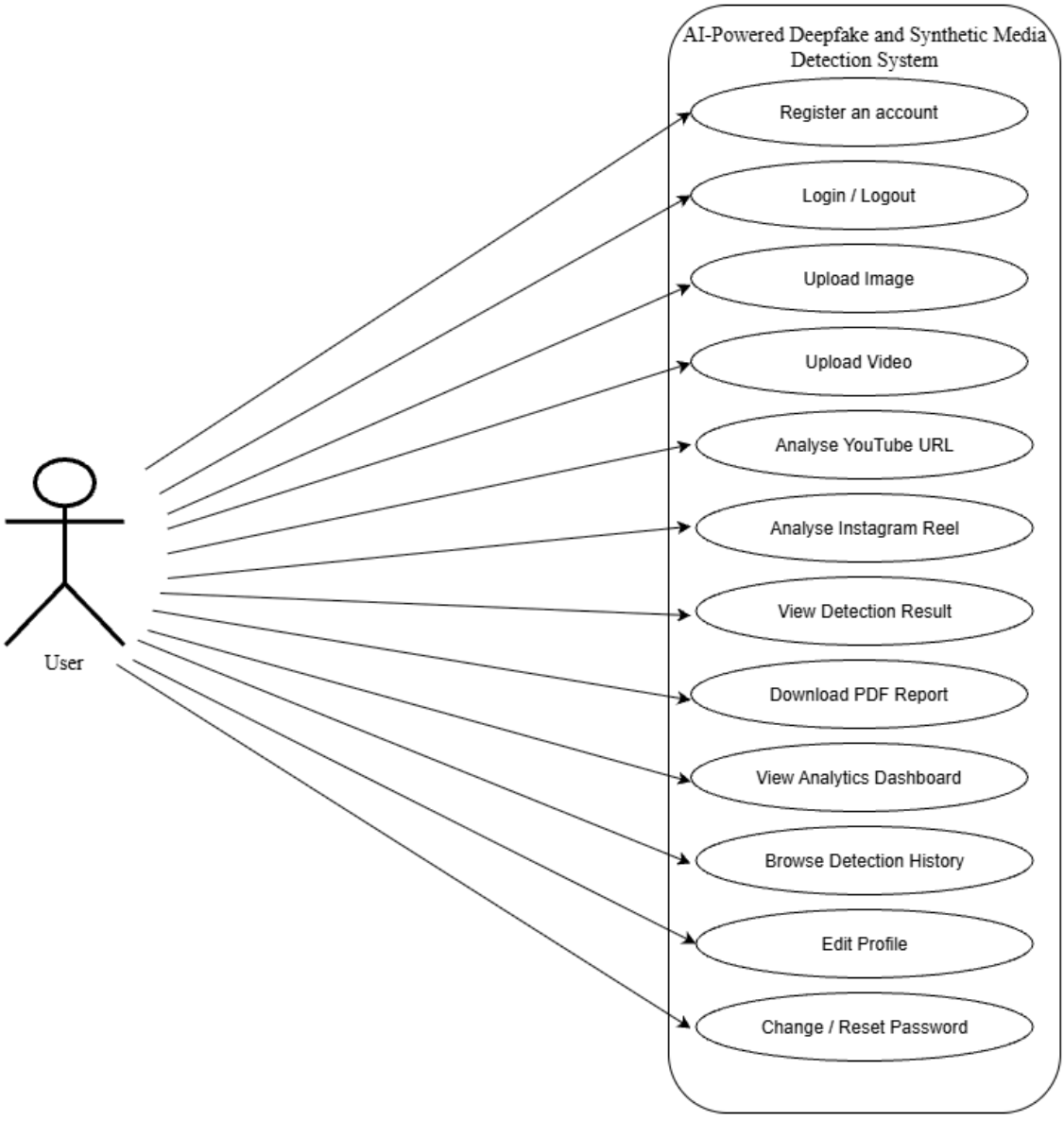


Figure 4.4: Use case diagram of the detection platform

4.3 CLASS DIAGRAM

The class diagram shows the persistent domain entities and the principal service classes. A report record links a detection to a generated PDF.

The service classes are not persisted.

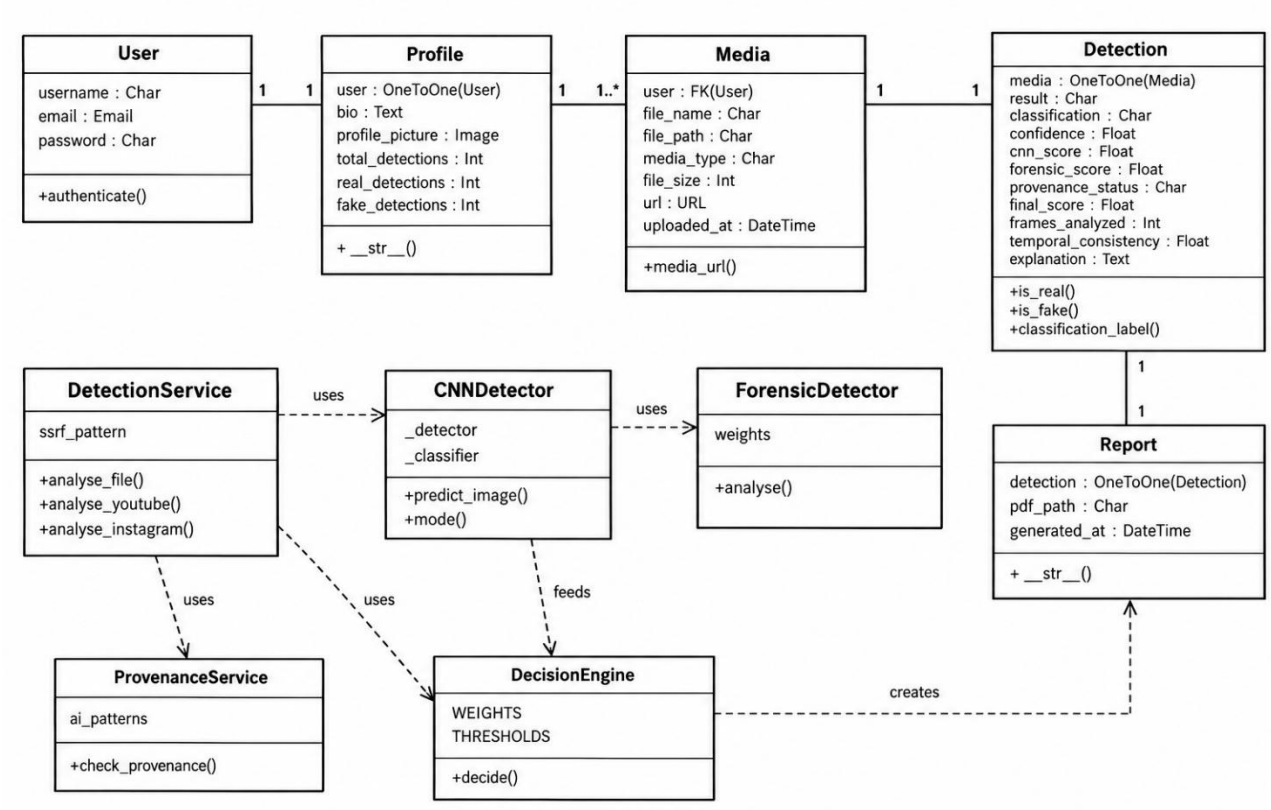


Figure 4.5: Class diagram of domain and service classes

4.4 STATE DIAGRAM

The state diagram traces the lifecycle of a single submission. Note that the uncertain state is not an error terminus: it flows into result stored like any other verdict, because declining to commit is a legitimate outcome that is recorded and reported in full.

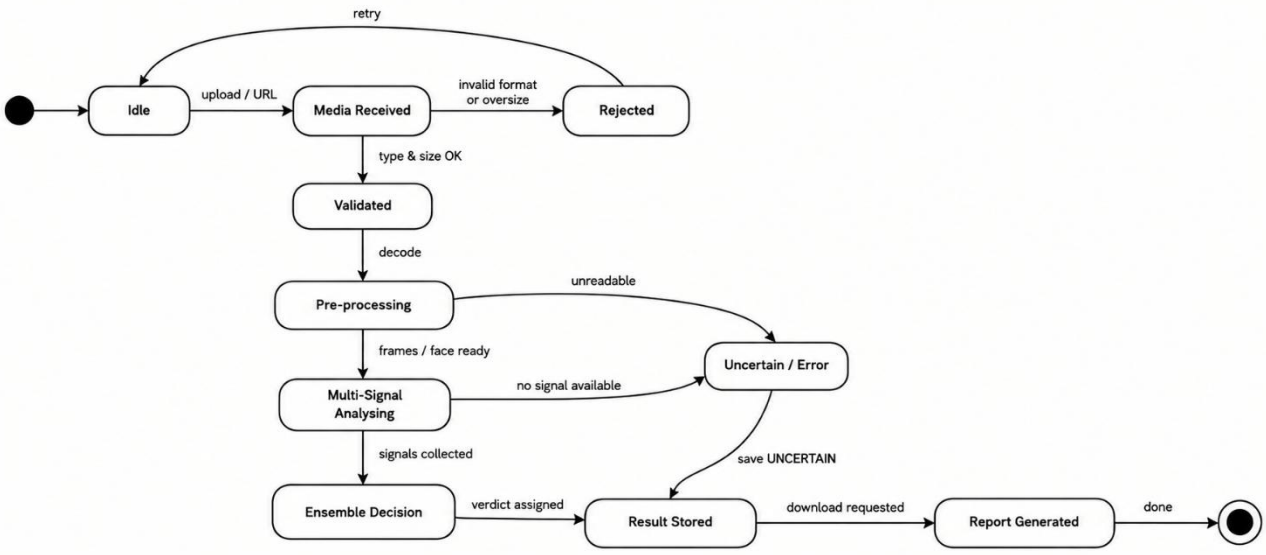


Figure 4.6: State diagram of a submission lifecycle

4.5 ACTIVITY DIAGRAM

This activity diagram traces the full detection flow from the user's side, including its key decision points. The result is saved and displayed, and a final decision determines whether a PDF report gets generated.

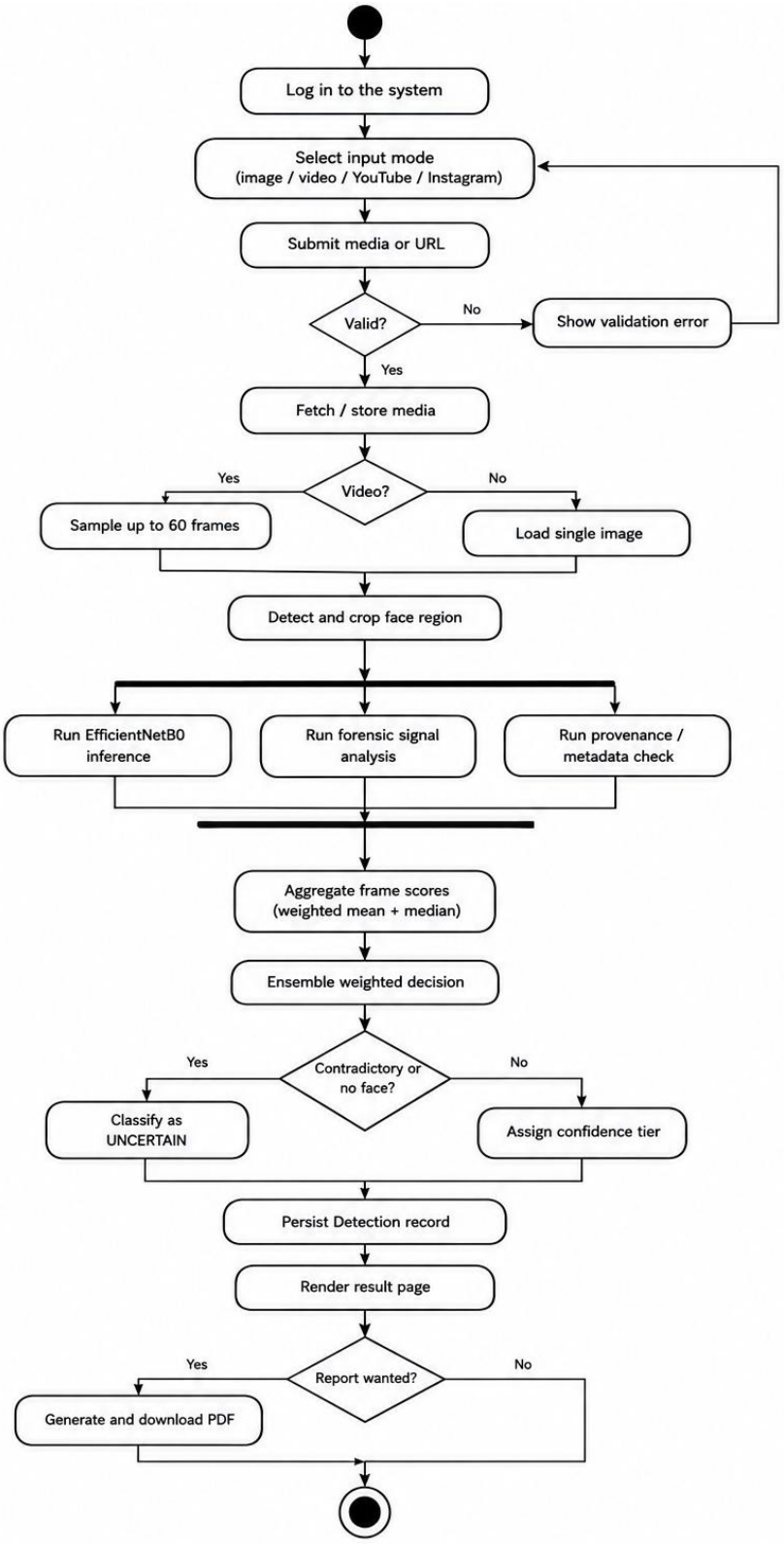


Figure 4.7: Activity diagram of the detection workflow

4.6 SEQUENCE DIAGRAM

This sequence diagram traces one detection request end-to-end. A separate later request from the user starts report creation and lets them download the PDF.

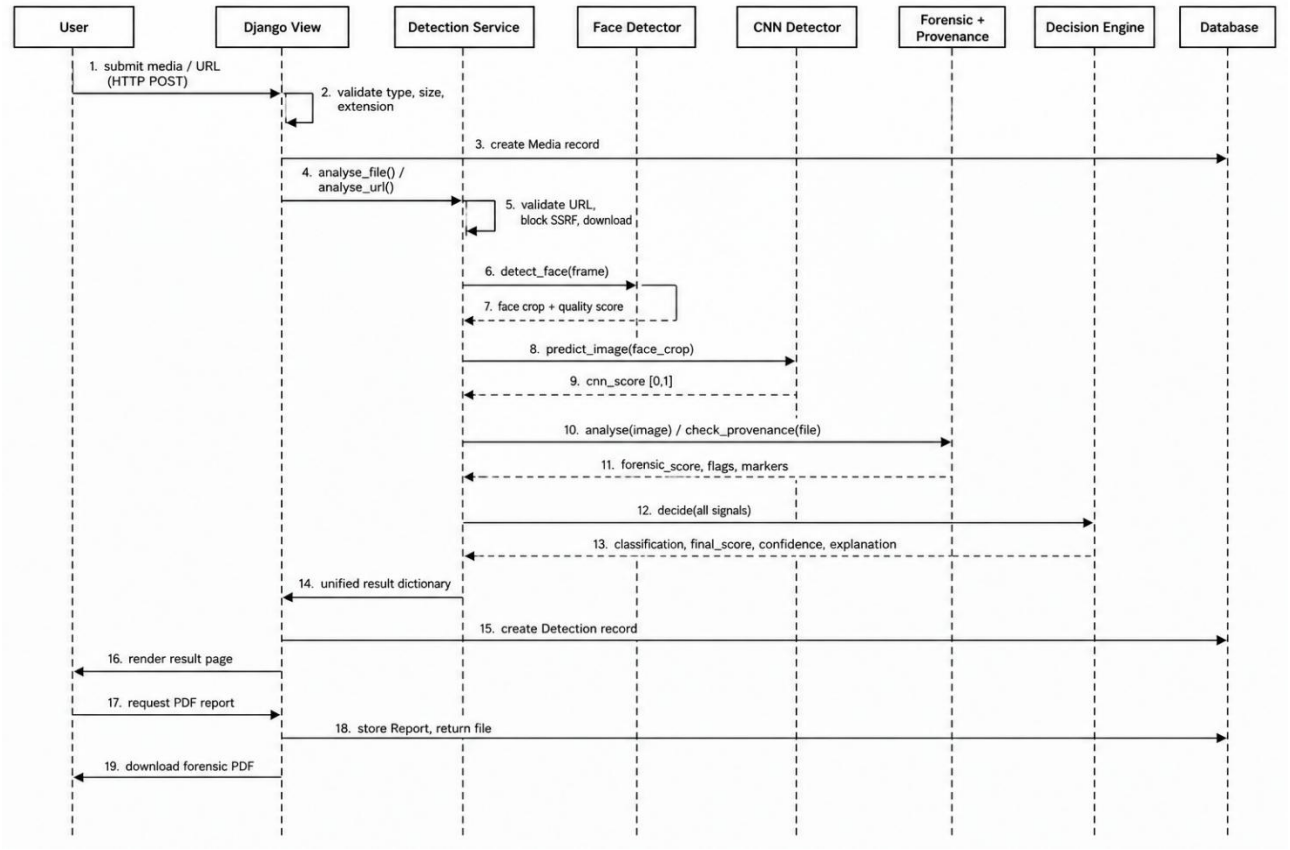


Figure 4.8: Sequence diagram of a single detection request

4.7 ENTITY RELATIONSHIP DIAGRAM

The ER diagram shows the database-level schema, distinct from the class diagram's object view. Deletes cascade (user → media → detection/report), while nullable foreign keys on Media and Detection let analysis rows survive cleanup operations.

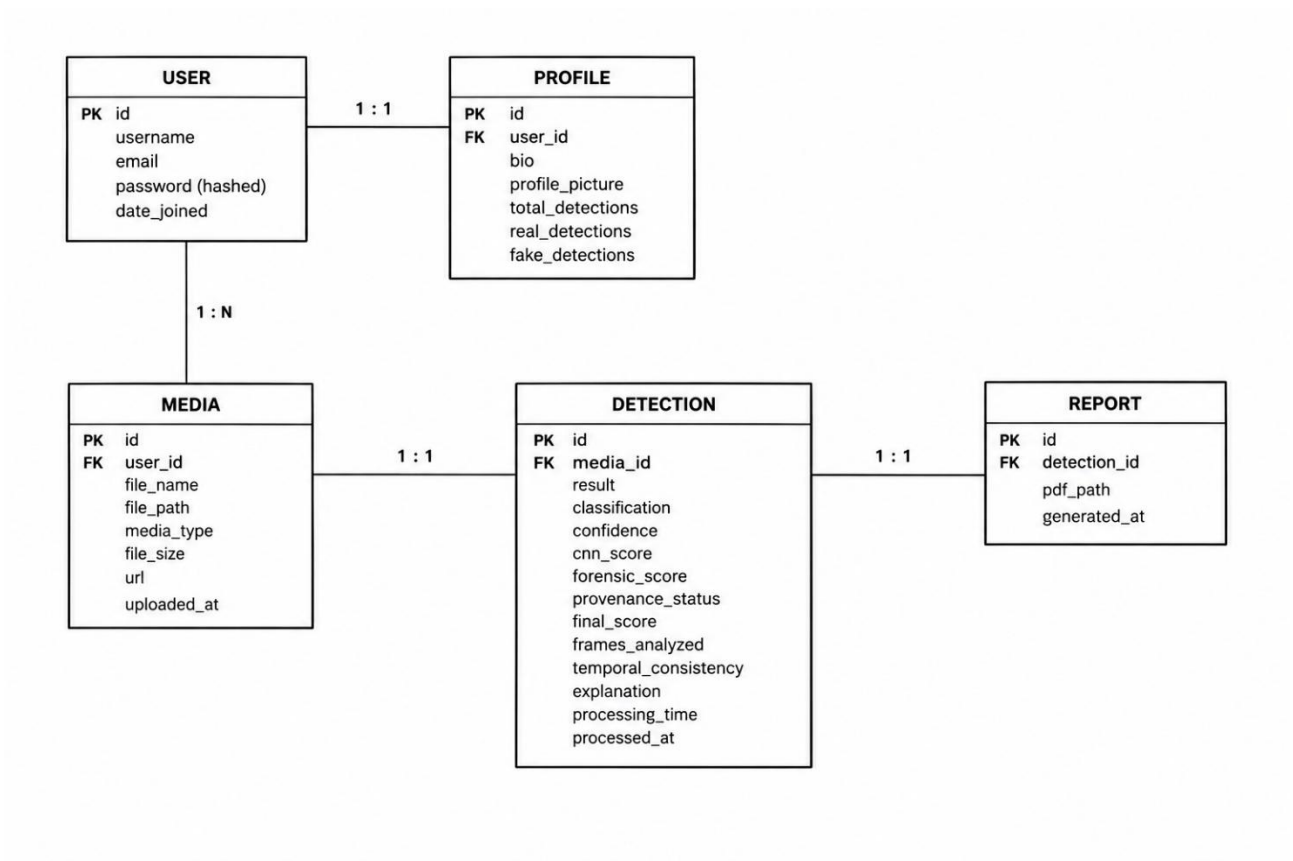


Figure 4.9: Entity relationship diagram of the database schema

CHAPTER 5

METHODOLOGY

This chapter describes how the system was built: the modules that compose it, the algorithms that drive the analysis, the layered architecture that organizes them, and a worked case study that traces a single submission from upload to report.

5.1 MODULE DESCRIPTIONS

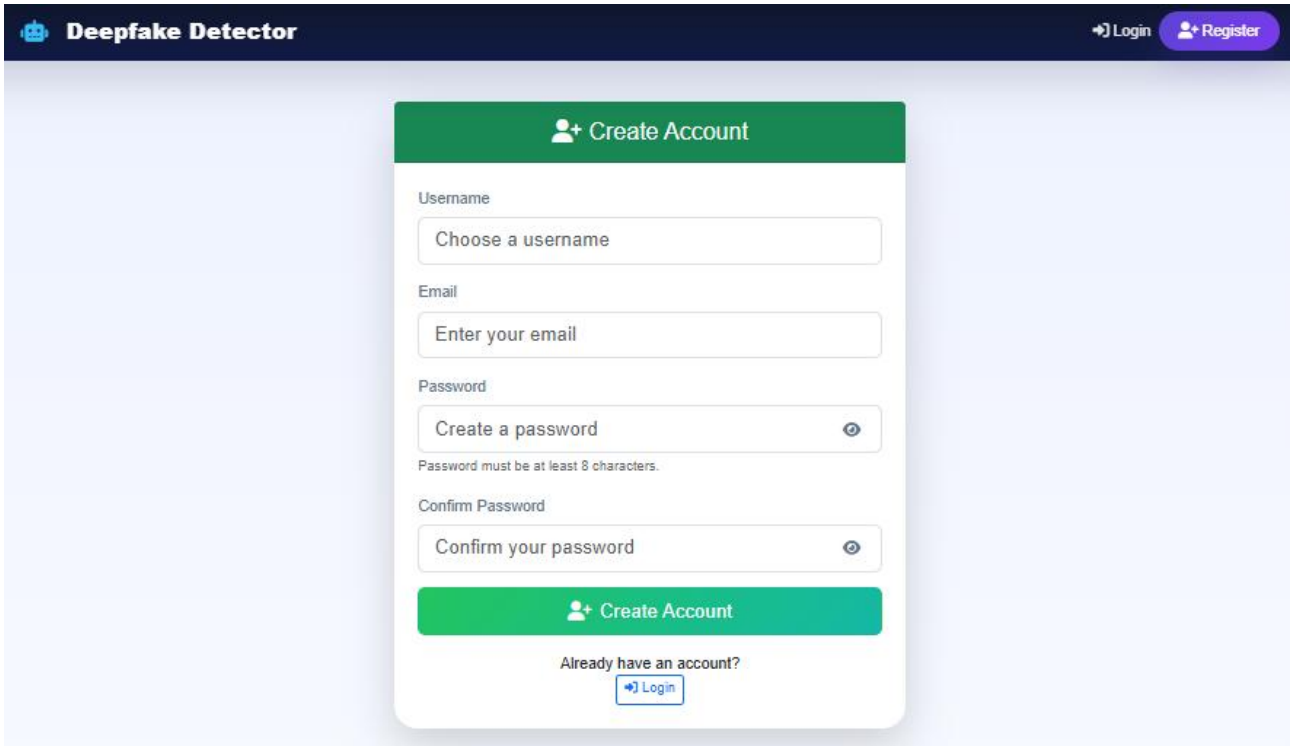
The application is organized into five Django applications and a set of detection modules that sit beneath them.

Table 5.1: Summary of implemented modules and responsibilities

Module	Location	Responsibility
Authentication and profile	users	Registration, login, logout, profile editing, password change and reset, per-user detection counters
Media intake and result presentation	detector.views	Request validation, file storage, record creation, result rendering, report delivery
Detection orchestration	detector.detection_service	Routing by media type, URL validation and SSRF blocking, external retrieval, temporary file cleanup
Face localisation	detector.face_detector	Haar cascade detection with DNN fallback, padding, quality scoring
Convolutional inference	detector.cnn_detector, detector.ml_model	Loading trained weights, pre-processing, fake probability estimation, heuristic fallback
Forensic analysis	detector.forensic_detector	ELA, DCT, noise uniformity, saturation and sharpness signals with weighted combination and flags
Provenance inspection	detector.provenance	EXIF, XMP, PNG text chunk and C2PA marker extraction and matching
Video analysis	detector.video_detector	Frame sampling, per-frame scoring, weighted aggregation, temporal consistency
Ensemble decision	detector.ensemble.decision_engine	Signal normalization, weighted fusion, guard conditions, tier assignment, explanation composition
Report generation	detector.utils	PDF forensic report construction and media handling utilities
Analytics	dashboard	Aggregate counts, distribution and seven-day trend data for charting
History and archive	history, reports	Per-user listing of past detections and generated reports

5.1.1 Authentication and profile module

The users application extends the framework authentication system with a profile model linked one-to-one with the user record. All detection routes are protected by a login requirement decorator, so an unauthenticated request is redirected rather than silently returning empty data.



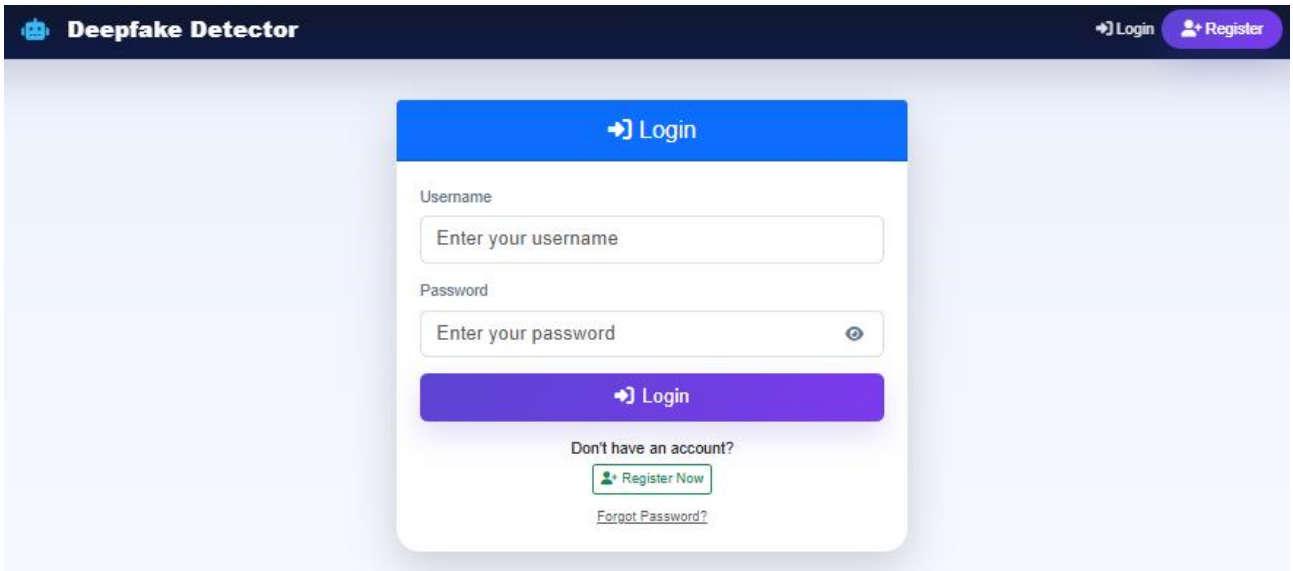


Figure 5.1: Registration and login interface

5.1.2 Media intake module

The upload view accepts a media type and a file, validates the extension against an allow-list and the size against a limit — ten megabytes for images and one hundred for video — and rejects anything

outside those bounds with a specific message before any bytes are written to disk. The two URL views apply the same pattern to a submitted link, with the additional URL validation described below.

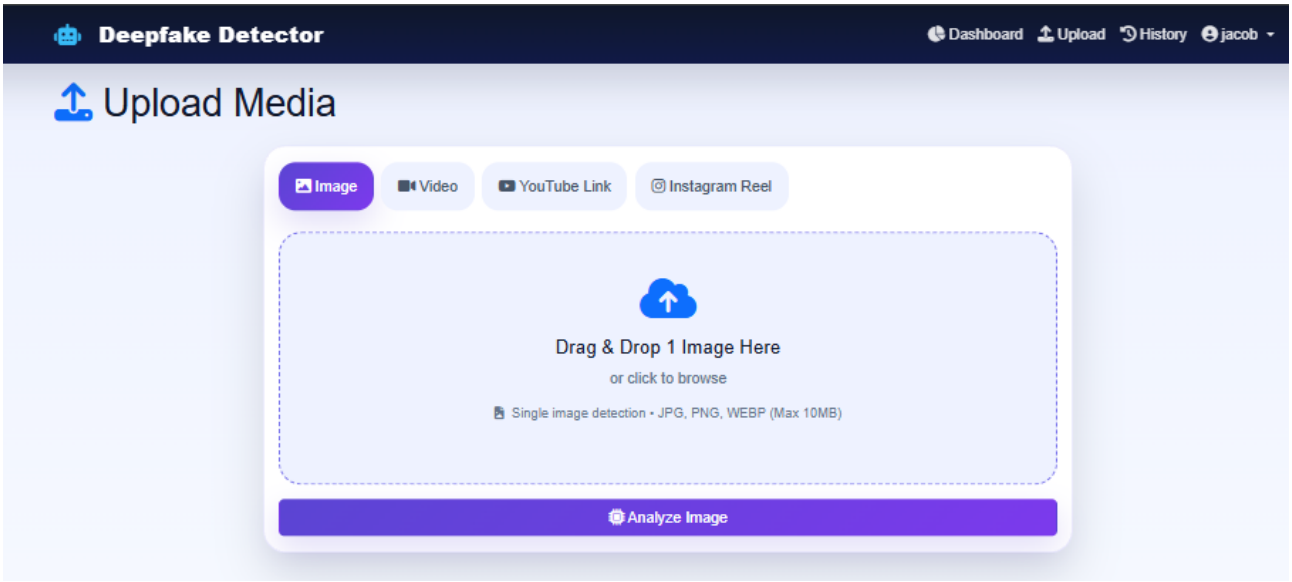


Figure 5.2: Media upload interface with channel selection

5.1.3 Detection orchestration module

Detection service: This service is the single-entry point for all views, routing requests to either the image or video detector based on media type. Errors are returned as structured results with user-facing messages; no placeholder or random score is ever substituted for a real one.

5.1.4 Face localization module

This module runs a Haar cascade classifier on the grayscale image, with a DNN-based detector as a fallback when available. Low-quality crops still get analyzed but are flagged, since small or blurry faces make network output less reliable.

5.1.5 Convolutional inference module

The inference module wraps the trained model in an interface that never throws errors back to the caller. Each result includes a status field, the current operating mode, and a flag indicating whether a face crop was actually used.

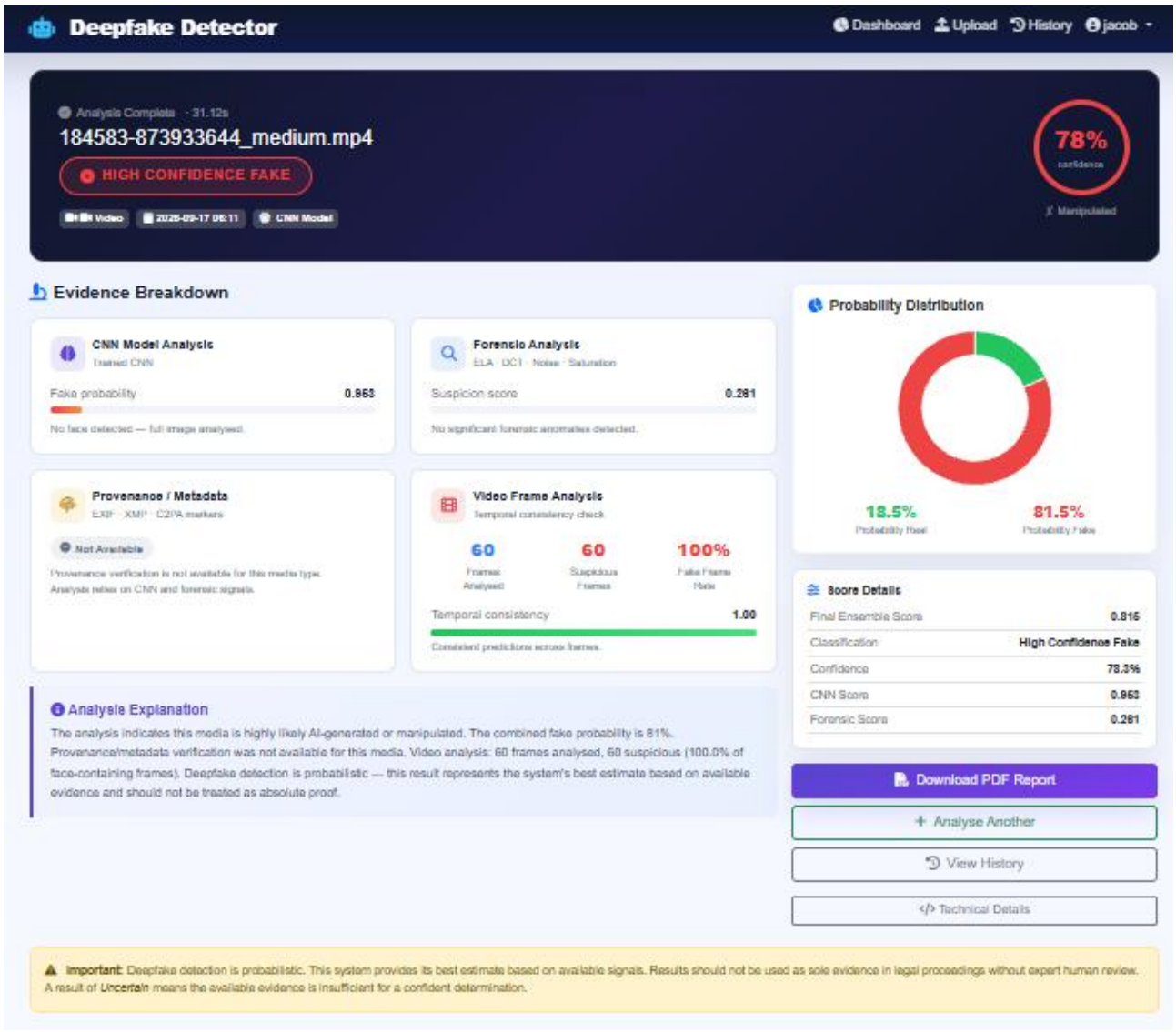


Figure 5.3: Detection result page with signal breakdown

5.1.6 Forensic analysis module

The forensic module computes five independent signals over the same image and combines them with fixed documented weights. The sharpness signal uses Laplacian variance to detect both the over-smoothing characteristic of generated faces and implausible over-sharpening.

Table 5.2: Forensic signal weights used in the analysis module

Signal	Technique	Weight	Rationale
Error level analysis	JPEG re-encode difference statistics	0.28	Most discriminative for splicing and inconsistent recompression
Sharpness	Laplacian variance	0.25	Reliable indicator of generative over-smoothing
Frequency	DCT high-frequency energy ratio	0.22	Captures periodic up sampling artefacts
Noise uniformity	Variance of local standard deviations	0.15	Secondary sensor-authenticity signal

Signal	Technique	Weight	Rationale
Saturation	HSV saturation mean and spread	0.10	Secondary color-profile signal

Each signal returns a value in the unit interval, and any signal that fails returns a neutral value rather than an extreme one, so that an analysis failure never pushes the verdict towards a false accusation.

5.1.7 Provenance inspection module

The provenance module reads text-valued metadata from the file container: EXIF software and creator fields, XMP creator and user-comment fields, PNG textual chunks that some tools use to record generation parameters, and JUMBF content-credential blocks embedded by C2PA-compliant tools. Unavailable and error indicate that the check could not be performed, in which case the signal is excluded from fusion entirely rather than contributing a neutral value that would dilute the others.

5.1.8 Video analysis module

Video analysis opens the file, reads the total frame count, and computes a stride so that the sampled frames are distributed uniformly across the whole timeline up to a cap of sixty. A video yielding fewer than three analysable frames returns an uncertain verdict rather than a guess.

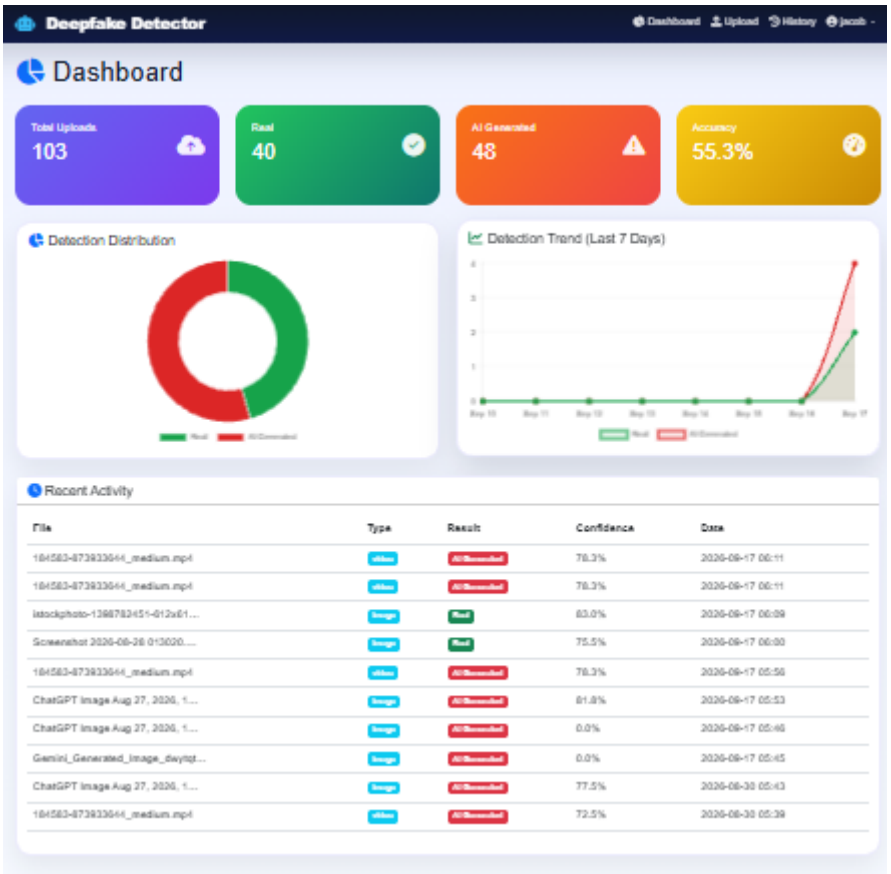


Figure 5.4: Analytics dashboard with distribution and trend charts

5.1.9 History, reports and analytics modules

The history application lists a user's past detections in reverse chronological order with the verdict, confidence and thumbnail of each. The reports application lists previously generated PDF documents and serves them for download. The dashboard application computes aggregate counts of real, fake and uncertain verdicts for the signed-in user, derives a simple quality indicator from the proportion of results exceeding seventy per cent confidence, and assembles labels and counts for the previous eight days, which the template renders as distribution and trend charts.

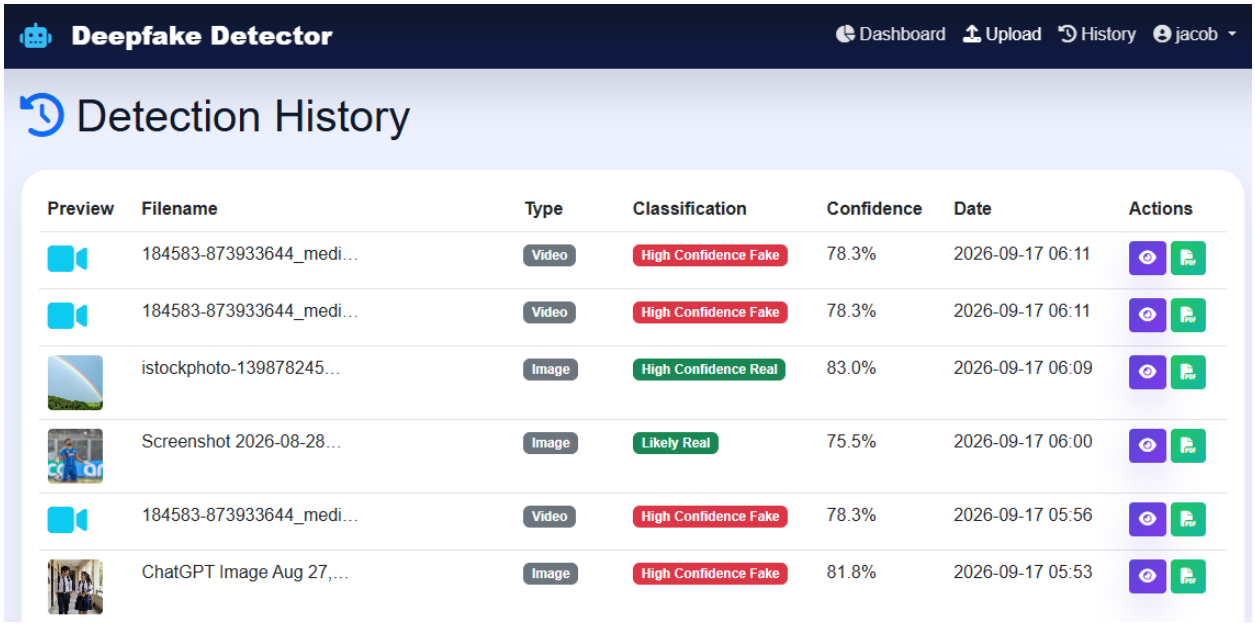


Figure 5.5: Detection history listing

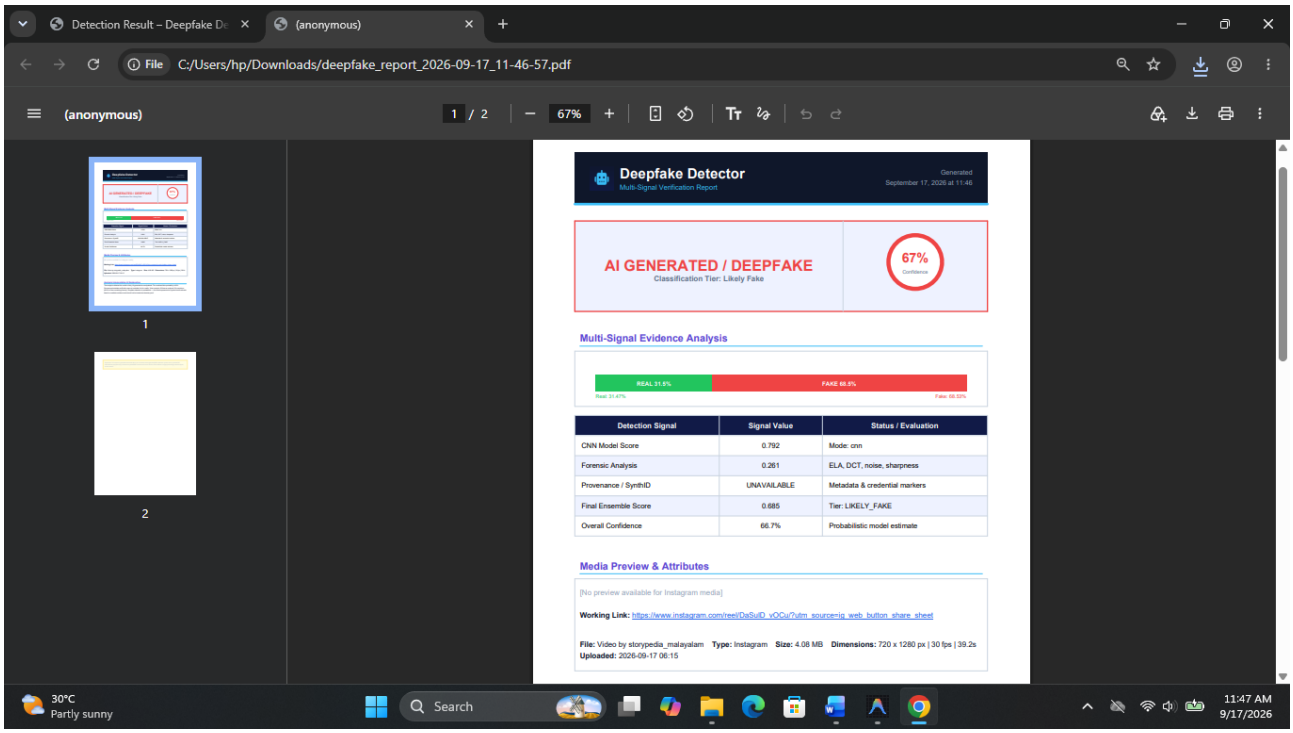


Figure 5.6: Generated PDF forensic report

5.1.10 Landing page and profile module

The landing page introduces the platform to an unauthenticated visitor, summarizes the four supported input channels and the nature of the analysis performed, and routes the visitor to registration or login.

The profile page presents the signed-in user with their picture, biography and the three detection counters, together with links to edit the profile and change the password.

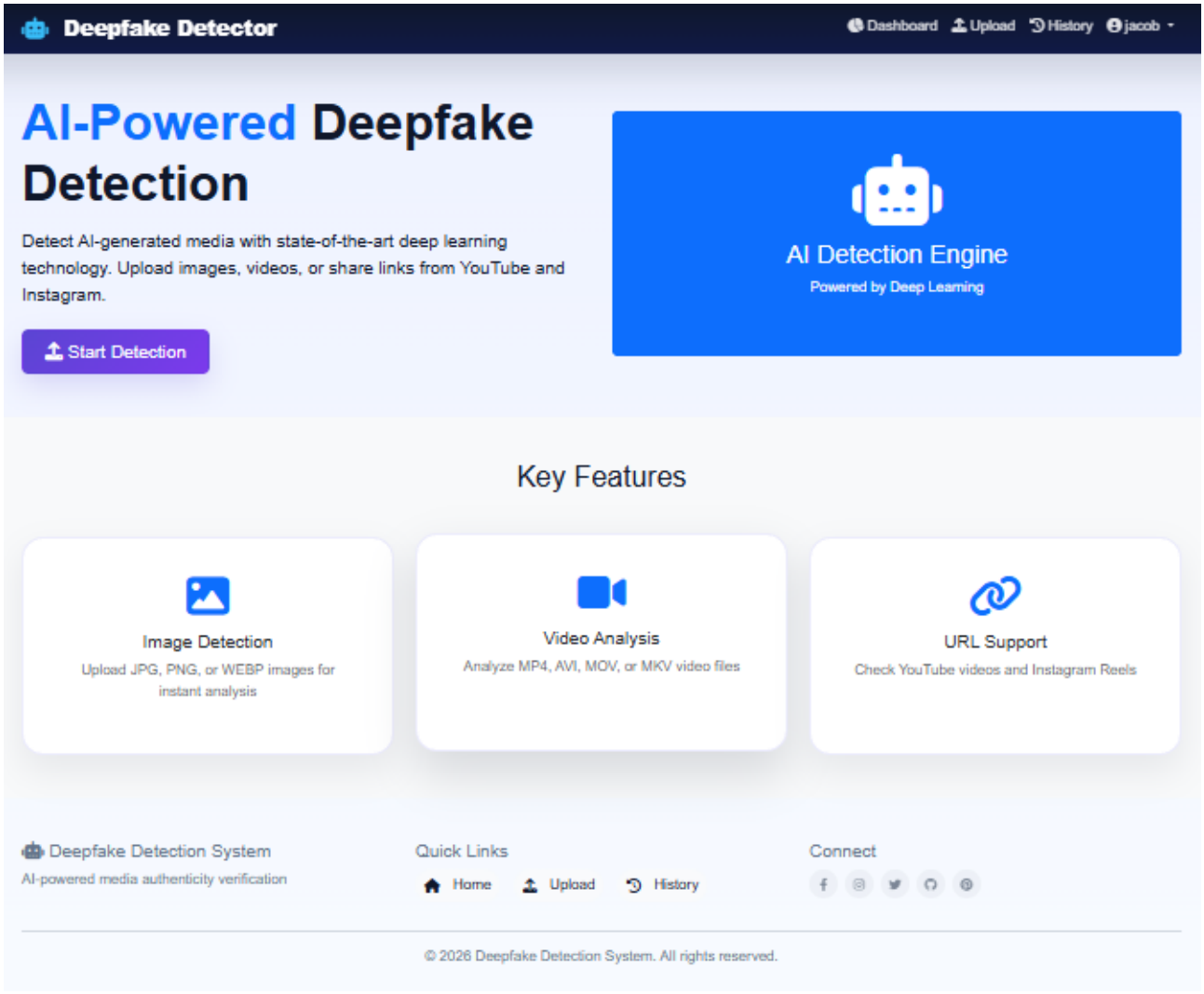


Figure 5.7: Landing page of the detection platform

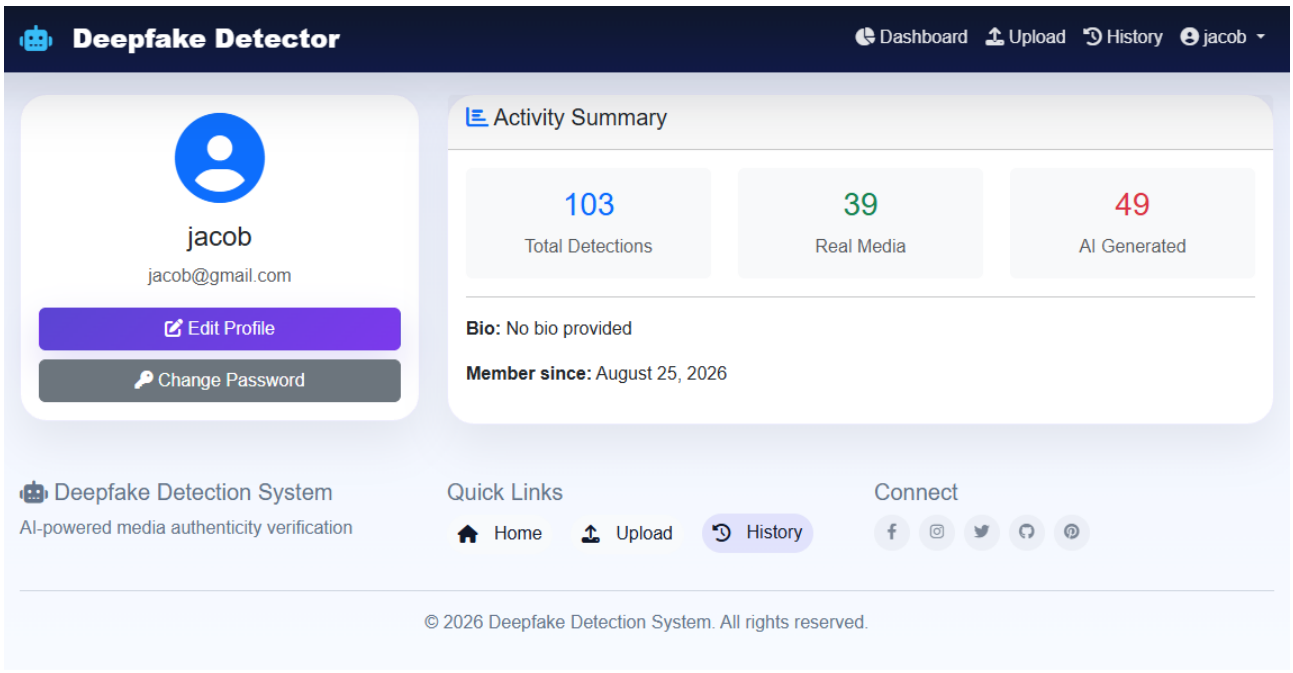


Figure 5.8: User profile page

5.2 ALGORITHMS

5.2.1 Two-stage transfer learning for the classifier

The classifier is obtained by transfer learning in two stages. Both stages use class-weighted binary cross-entropy, monitor the area under the receiver operating characteristic curve for checkpointing and early stopping, and reduce the learning rate on plateau.

```
ALGORITHM 1: Two-stage training
INPUT  : dataset directories train/{real,fake}, validation/{real,fake}
OUTPUT : trained weights file

1.  base <- EfficientNetB0(weights = imagenet, include_top = false)
2.  base.trainable <- false
3.  head <- GAP -> BatchNorm -> Dense(512, relu) -> Dropout(0.5)
              -> Dense(256, relu) -> Dropout(0.3) -> Dense(1, sigmoid)
4.  model <- compose(base, head)
5.  compile(model, Adam(lr = 1e-3), binary_crossentropy,
            metrics = {accuracy, precision, recall, auc})
6.  gen   <- augment(rotation 25, shift 0.15, shear 0.15, zoom 0.20,
                     hflip, brightness [0.75,1.25], channel shift 20,
                     simulated JPEG quality [65,95])
7.  w     <- balanced_class_weights(train_labels)
8.  fit(model, gen, epochs = 25, class_weight = w,
        callbacks = {EarlyStopping(auc, patience 8),
                     ReduceLROnPlateau(factor 0.3, patience 4),
                     ModelCheckpoint(stage1_best)})

9.  base.trainable <- true
10. FOR each layer in base.layers[0 .. len-30]  DO  layer.trainable <- false
11. compile(model, Adam(lr = 1e-5), binary_crossentropy, same metrics)
12. fit(model, gen, epochs = 20, class_weight = w,
        callbacks = {EarlyStopping, ReduceLROnPlateau,
                     ModelCheckpoint(stage2_best)})
13. save(model)
```

5.2.2 Ensemble decision algorithm

The decision engine is the most consequential algorithm in the system because it converts several numbers into a claim about reality. When it is, the network dominates; when the system has fallen back to heuristics, the forensic signals are given greater relative weight because the heuristic substitute is far less reliable.

Table 5.3: Ensemble weights for active-CNN and heuristic modes

Signal	CNN mode with provenance	CNN mode without provenance	Heuristic mode with provenance	Heuristic mode without provenance
Convolutional network	0.75	0.80	0.35	0.45
Forensic analysis	0.15	0.20	0.45	0.55
Provenance	0.10	—	0.20	—

Table 5.4: Classification tiers and score thresholds

Tier	Fused fake probability	Interpretation
High Confidence Real	below 0.20	Strong convergent evidence of authenticity
Likely Real	0.20 to 0.40	Evidence leans authentic with moderate confidence
Uncertain	0.40 to 0.60, or guard triggered	Evidence insufficient or contradictory; no verdict issued
Likely Fake	0.60 to 0.80	Evidence leans manipulated with moderate confidence
High Confidence Fake	above 0.80	Strong convergent evidence of manipulation

```
ALGORITHM 2: Ensemble decision
INPUT  : cnn_result, forensic_result, provenance_result,
         video_result (optional), face_found, cnn_mode
OUTPUT : classification, fused score, confidence, explanation

1.  scores <- empty ; weights <- empty
2.  IF cnn_result.status = ok THEN append(cnn_result.score, w_cnn)
3.  IF forensic_result.status = ok THEN append(forensic.score, w_for)
4.  IF provenance.status = detected      THEN append(0.90, w_prov)
    ELSE IF provenance.status = not_detected THEN append(0.25, w_prov)
    ELSE provenance is excluded from fusion
5.  IF scores is empty THEN RETURN UNCERTAIN("no signals available")
6.  final <- sum(score_i * weight_i) / sum(weight_i)

7.  contradictory <- (cnn_mode = heuristic) AND
                     |cnn.score - forensic.score| > 0.35
8.  face_penalty  <- (NOT face_found) AND (cnn_mode = heuristic)

9.  tier <- threshold_tier(final)          // Table 5.4
10. IF cnn_mode = cnn THEN                 // neural safety override
      IF cnn.score >= 0.60 AND tier in {HCR, LR, UNC} THEN tier <- LIKELY_FAKE
      ELSE IF cnn.score >= 0.45 AND tier in {HCR, LR} THEN tier <- UNCERTAIN
11. IF contradictory OR face_penalty THEN tier <- UNCERTAIN

12. confidence <- map_to_bounded_range(final, tier)   // capped at 95
13. IF video_result exists AND temporal_consistency < 0.50 THEN
      moderate confidence and append temporal note
14. explanation <- compose(tier, final, signal breakdown, flags,
```

```
                                frame statistics, standing caution)
15. RETURN (tier, final, confidence, explanation)
```

Two aspects of this algorithm deserve emphasis. Conversely, the guards in step eleven can only move a verdict towards uncertainty, never away from it, so the system is structurally incapable of manufacturing confidence it does not have.

5.2.3 Video frame aggregation algorithm

```
ALGORITHM 3: Video frame aggregation
INPUT  : video file path, max_frames = 60
OUTPUT : fused fake score, temporal consistency, frame statistics

1.  total   <- frame count of the video
2.  stride  <- max(1, total / max_frames)
3.  results <- empty list
4.  FOR each frame index i in the video DO
5.      IF i mod stride = 0 AND |results| < max_frames THEN
6.          face <- detect_face(frame_i)
7.          score <- cnn_predict(frame_i, face.crop if face.found)
8.          append (score, face.found) to results
9.  IF |results| < 3 THEN RETURN UNCERTAIN("insufficient frames")
10. FOR each r in results: w_r <- 2.0 if r.face_found else 1.0
11. weighted <- sum(r.score * w_r) / sum(w_r)
12. median   <- median(all scores)
13. fused    <- 0.65 * weighted + 0.35 * median
14. consistency <- clip(1.0 - 2.0 * stdev(all scores), 0, 1)
15. suspicious  <- count(score >= 0.55)
16. fake_pct    <- suspicious among face frames / face frames * 100
17. RETURN (fused, consistency, |results|, suspicious, fake_pct)
```

5.2.4 Forensic signal analysis algorithm

The forensic analyzer is the second source of evidence and operates without any knowledge of the network output, which is what makes its agreement or disagreement informative.

```
ALGORITHM 4: Forensic signal analysis
INPUT  : bgr_image
OUTPUT : combined forensic score, individual scores, observation flags

1.  IF image is empty THEN RETURN error result with neutral score 0.40
2.  gray <- greyscale(bgr_image)

3.  // Signal 1: error level analysis
4.  recompressed <- jpeg_decode(jpeg_encode(bgr_image, quality = 90))
5.  diff <- |bgr_image - recompressed|
6.  ela <- 0.70 if mean(diff) > 30 or stdev(diff) > 25
           0.45 if mean(diff) > 15 or stdev(diff) > 12
           0.25 otherwise

7.  // Signal 2: frequency-domain energy ratio
8.  D <- DCT(resize(gray, 64 x 64))
9.  hf_ratio <- 1 - sum(D[0..8, 0..8]^2) / sum(D^2)
10. dct <- 0.70 if hf_ratio > 0.60, 0.45 if > 0.45, else 0.25

11. // Signal 3: local noise uniformity
12. partition gray into a 4 x 4 grid of patches
13. s <- stdev over patches of stdev(patch)
14. noise <- 0.65 if s < 3.0, 0.40 if s < 8.0, else 0.25
```

```
15. // Signal 4: colour saturation profile
16. sat <- saturation channel of HSV(bgr_image)
17. saturation <- 0.65 if mean(sat) > 180 or stdev(sat) < 15
                  0.60 if mean(sat) < 20
                  0.28 otherwise

18. // Signal 5: sharpness signature
19. v <- variance(Laplacian(gray))
20. sharp <- 0.78 if v < 30, 0.60 if v < 80,
             0.28 if v < 500, else 0.20

21. combined <- 0.28*ela + 0.25*sharp + 0.22*dct
                + 0.15*noise + 0.10*saturation
22. flags <- names of all signals exceeding 0.55
23. RETURN (clip(combined, 0, 1), individual scores, flags)
```

The threshold constants in this algorithm were chosen by reasoning from the ranges reported in the forensic literature and by observation of the values produced on known real and known generated images during development. They are not learned, and section 7.2.6 records this as a limitation. They are, however, gathered in one module with the reasoning documented alongside each, so that recalibrating the analyzer against a labelled corpus would be a matter of adjusting constants rather than rewriting logic.

5.3 ARCHITECTURE

The system follows a four-layer architecture with a clear dependency direction: each layer depends only on the layer beneath it, and no layer reaches upward.

5.3.1 Presentation layer

The presentation layer consists of Django templates rendered with Bootstrap 5 for responsive layout, together with Chart.

5.3.2 Application layer

The application layer holds the Django views, forms and URL routing across the five applications.

5.3.3 Intelligence layer

The intelligence layer contains the face detector, the convolutional detector, the forensic detector, the provenance service, the video analyzer and the ensemble decision engine.

5.3.4 Data layer

The data layer comprises the SQLite database holding user, profile, media, detection and report records, a media store on the file system holding uploads, generated reports and profile pictures, and a model store holding the trained weight files.

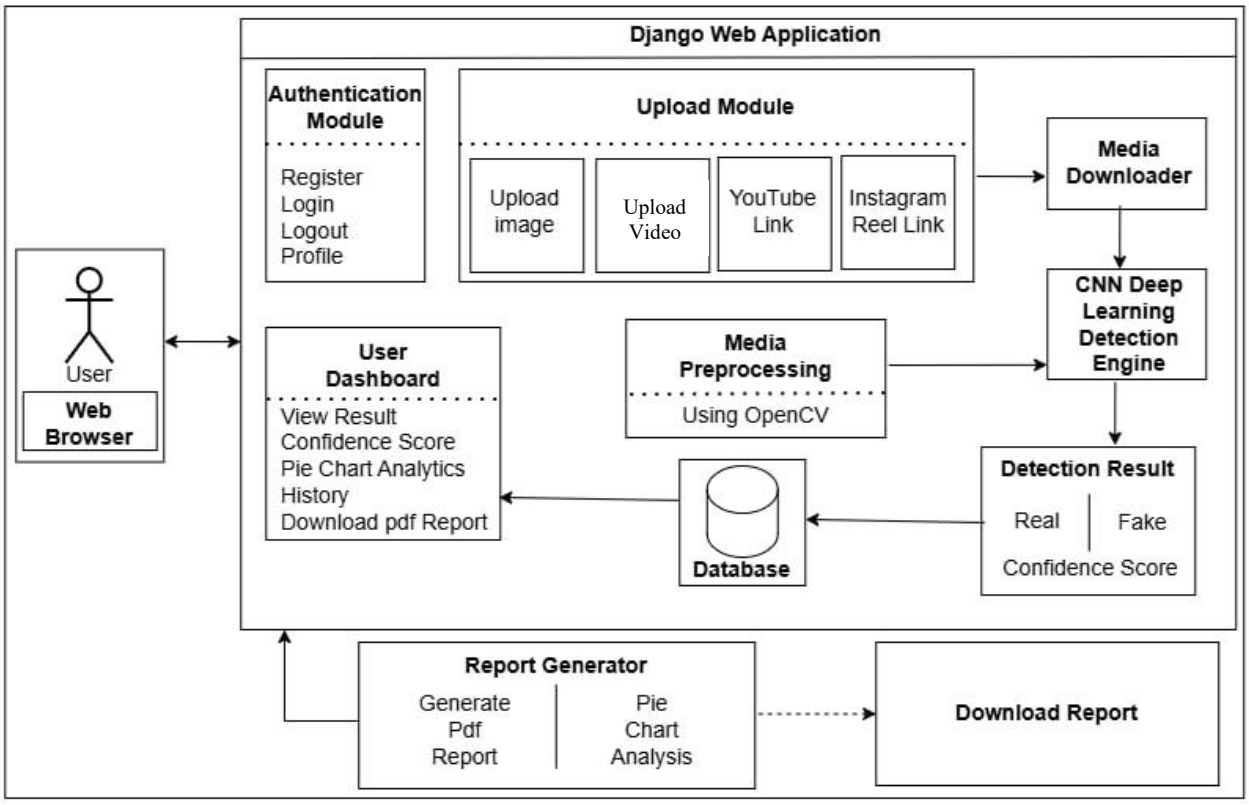


Figure 5.9: System architecture of the detection platform

5.4 CASE STUDY

To make the pipeline concrete, this section traces a single submission end to end.

5.4.1 Submission and validation

The user selects the video channel on the upload page and chooses a file of roughly seven megabytes with an MP4 extension.

5.4.2 Frame sampling

The detection service routes the request to the video analyzer, which opens the file and reports its dimensions, frame rate, frame count and duration.

5.4.3 Per-frame analysis and aggregation

Face localization succeeds on the majority of sampled frames; the remainder are cutaways or motion-blurred and yield no face. The count of frames exceeding the suspicion threshold, and the proportion of those among face-bearing frames, are recorded.

5.4.4 Forensic and provenance analysis

A representative frame is drawn from the middle of the clip and subjected to the five forensic tests.

5.4.5 Fusion and verdict

The three signals are handed to the decision engine together with the video aggregation. An explanation is composed naming the fused probability, the forensic flags raised, the number of frames analyzed and the number found suspicious, and closing with the standing caution that detection is probabilistic.

5.4.6 Persistence and reporting

A detection record is written carrying the verdict, the tier, the confidence, the individual network and forensic scores, the provenance status, the frame statistics, the temporal consistency, the fused score, the explanation and the elapsed processing time. The user then requests a report; a PDF is generated containing the same information in document form, a report record is created, and the file is served as a download.

CHAPTER 6

IMPLEMENTATION DETAILS

6.1 IMPLEMENTATION PROCEDURES

6.1.1 Development environment

Development was carried out in Python 3. A setup script automates environment creation, dependency installation, database migration and superuser creation for a first-time installation. Using a single script for first-time installation removes a common source of failure in student projects, where a result cannot be reproduced because a step was forgotten or a library version differs.

6.1.2 Project scaffolding and data model

The Django project was created with five applications registered alongside the framework defaults and the crispy-forms, Bootstrap 5 and REST framework packages. This backward-compatible approach is visible in the model definition, where the legacy fields are grouped separately from the multi-signal additions. Registering the applications explicitly and keeping each one's models, views and templates together follows the framework's conventions, which makes the structure familiar to any Django developer who inherits the project.

6.1.3 Dataset preparation

The dataset directory was organized into train, validation and test splits, each containing a real and a fake subdirectory, which is the layout the Keras directory-based data generator expects. A verification utility walks the tree and reports the count in each class so that imbalance is detected before training rather than after. The splits were kept exactly balanced between the two classes, which removes class imbalance as a confounding variable, although balanced class weights are still computed and applied during training as a safeguard. Detecting imbalance before training is cheap, whereas discovering it afterwards would require retraining and could invalidate comparisons between runs.

6.1.4 Model training implementation

The training script implements the two-stage schedule described in Chapter 5. A separate evaluation script loads the saved weights, runs them over the held-out test split and reports accuracy, precision, recall and the confusion matrix. Separating training from evaluation means that the held-out split is touched only once, after the model has been finalised, which protects the reported figures from accidental tuning against the test data.

6.1.5 Detection pipeline implementation

The detection modules were implemented bottom-up so that each could be tested in isolation before the layer above it existed. The prohibition on random fallback values is stated explicitly in the source of both the detection service and the views. Building bottom-up also meant that a failure could be traced to a specific module rather than to the interaction of several untested parts.

6.1.6 Web layer and security implementation

Views were kept deliberately thin. Debug mode is controlled by configuration and must be disabled for any real deployment, a point noted in the limitations. Keeping views thin means that validation, authorisation and rendering are the only responsibilities of the web layer, while every decision about media is made in the detection layer where it can be tested without a browser.

6.1.7 Report generation implementation

PDF generation uses ReportLab to compose a document containing a header, the submission metadata, the verdict with its tier and confidence, a table of the individual signal scores, the list of forensic flags, the frame statistics where applicable, the written explanation and a closing disclaimer. The closing disclaimer repeats the standing caution that detection is probabilistic, so that the report remains responsibly worded even when it is separated from the application that generated it.

6.1.8 Database schema implementation

The schema was generated from the model definitions through the framework migration system rather than written by hand, so that the Python definition remains the single source of truth and every change is captured as a reviewable migration file. Reviewable migration files also give a history of how the schema evolved, which is helpful when the project is extended or moved to a different database engine.

Table 6.1: Application database tables and key columns

Table	Key columns	Relationship
auth_user	id, username, email, password hash, date_joined	Framework-supplied; parent of profile and media
users_profile	id, user_id, bio, profile_picture, total_detections, real_detections, fake_detections	One-to-one with auth_user
detector_media	id, user_id, file_name, file_path, media_type, file_size, url, uploaded_at	Many-to-one with auth_user
detector_detection	id, media_id, result, classification, confidence, cnn_score, forensic_score, provenance_status, provenance_markers, final_score, frames_analyzed, suspicious_frames, fake_frame_pct, temporal_consistency, explanation, processing_time,	One-to-one with detector_media

Table	Key columns	Relationship
	source_type, forensic_flags, face_found, cnn_mode, processed_at	
detector_report	id, detection_id, pdf_path, generated_at	One-to-one with detector_detection

Two implementation details in this schema are worth drawing out. This was a considered trade: the lists are short, are always read as a whole and are never queried by their contents, so normalising them into separate tables would have added two joins to every result page for no benefit. The alternative design remains straightforward to adopt later if querying by those fields ever becomes necessary, because the migration framework can transform the stored values into related tables.

6.1.9 Installation and deployment procedure

The application is installed and started with the sequence below.

```
# 1. Obtain the source and enter the project directory
git clone https://github.com/Alanbenny05/deepfake-detection-system.git
cd deepfake-detection-system

# 2. Create and activate an isolated environment
python -m venv venv
venv\Scripts\activate            # Windows
source venv/bin/activate         # Linux / macOS

# 3. Install the pinned dependency set
pip install -r requirements.txt

# 4. Create the local configuration from the template
copy .env.example .env           # Windows
cp .env.example .env             # Linux / macOS
#    then edit .env to set SECRET_KEY and DEBUG

# 5. Create the database schema
python manage.py makemigrations
python manage.py migrate

# 6. Verify that the dataset directories are correctly populated
python check_dataset.py

# 7. Train the detector (optional if weights are already present)
python train_model.py            # two-stage transfer learning
python test_trained_model.py     # evaluate on the held-out split

# 8. Start the development server
python manage.py runserver
#    then open http://127.0.0.1:8000/ in a browser
```

Step seven is optional because the trained weight files are distributed with the project. This behaviour was verified during testing by renaming the weights directory and confirming that submissions still produced reasoned, correctly caveated verdicts rather than errors. The same renaming test was valuable because it exercised the fallback path under realistic conditions instead of relying on a simulated failure inside a unit test.

6.1.10 Source organization

The delivered source tree is organized so that the boundary between the web layer and the detection layer is visible from the directory structure alone. This separation also allows the detection layer to be reused behind a different front end, such as a command-line tool or a service interface, without copying code out of the web application.

```
deepfake-detection-system/
  deepfake_detection/       core project configuration
    settings.py             apps, middleware, database, media paths
    urls.py                 root URL routing
    wsgi.py / asgi.py       server entry points
  users/                    authentication and profile application
    models.py               Profile model
    forms.py                registration and update forms
    views.py                register, login, profile, password change
    tests.py                authentication test suite
  detector/                 detection application
    models.py               Media, Detection, Report models
    views.py                thin views; no detection logic
    forms.py                upload and URL forms
    detection_service.py    routing, URL validation, retrieval, cleanup
    face_detector.py        Haar cascade with DNN fallback
    cnn_detector.py         wrapper over the trained network
    ml_model.py             model loading and inference
    forensic_detector.py    ELA, DCT, noise, saturation, sharpness
    image_detector.py       single-image pipeline
    video_detector.py       sampling, per-frame scoring, aggregation
    ensemble/
      decision_engine.py    weights, thresholds, guards, explanation
    provenance/
      provenance_service.py provenance orchestration
      synthid_detector.py   EXIF, XMP, PNG chunk, C2PA matching
    utils.py                PDF report generation and helpers
    tests.py                detection test suite
  dashboard/                analytics aggregation and charting data
  history/                  per-user detection listing
  reports/                  generated report archive
  templates/                Bootstrap 5 templates
  static/                   CSS, JavaScript and images
  dataset/                  train / validation / test splits
  models/                   trained weight files (.h5)
  media/                    uploads, reports, profile pictures
  logs/                     application log
  train_model.py            two-stage training script
  evaluate_model.py         held-out evaluation
  check_dataset.py          dataset structure verification
  requirements.txt          pinned dependencies
  manage.py                 framework command-line entry point
```

6.2 TESTING METHODS AND RESULTS

6.2.1 Dataset used for evaluation

The model was trained and evaluated on a balanced corpus of facial images divided into three disjoint splits. It must be stated plainly that a corpus of this size is small by the standards of published detection research, and that performance on generation techniques not represented in it should be expected to be lower — this is precisely the cross-distribution degradation documented in the literature reviewed in Chapter 3. Because the three splits are disjoint, no image used to measure performance was seen during training or used to select a checkpoint, which keeps the held-out measurements honest as far as the dataset itself allows.

Table 6.2: Dataset composition and split

Split	Real images	Fake images	Total	Proportion
Training	607	607	1,214	75.6 %
Validation	132	132	264	16.4 %
Test	124	124	248	8.0 %
Total	863	863	1,726	100 %

6.2.2 Testing strategy

Four complementary levels of testing were applied.

- **Unit testing.**
- **Integration testing.**
- **Decision engine testing.**
- **System and security testing.**

6.2.3 Test suite results

Table 6.3: Unit and integration test suite results

Test class	Test case	Expected behavior	Result
FaceDetectorTests	test_no_face_detected	Returns found = false without raising on a blank image	Pass
FaceDetectorTests	test_empty_image	Handles a zero-size array gracefully	Pass
ForensicTests	test_forensic_analysis_structure	Returns all five signal keys plus status and flags	Pass
ForensicTests	test_empty_image	Returns error status with a neutral score	Pass
ProvenanceTests	test_nonexistent_file	Returns error status rather than raising	Pass
ProvenanceTests	test_standard_image_no_markers	Returns not_detected for a plain image	Pass
DecisionEngineTests	test_high_confidence_fake	Converging high scores yield HIGH_CONFIDENCE_FAKE	Pass
DecisionEngineTests	test_high_confidence_real	Converging low scores yield HIGH_CONFIDENCE_REAL	Pass
DecisionEngineTests	test_contradictory_signals_yields_uncertain	Divergent signals yield UNCERTAIN	Pass

Test class	Test case	Expected behavior	Result
DecisionEngineTests	test_no_signals_yields_uncertain	Absent signals yield UNCERTAIN	Pass
URLValidationTests	test_valid_youtube_urls	Accepts youtube.com and youtu.be hosts	Pass
URLValidationTests	test_invalid_youtube_urls	Rejects unrelated hosts with a message	Pass
URLValidationTests	test_ssrf_blocked_ips	Rejects loopback and private ranges	Pass
IntegrationTests	test_upload_image_flow	Upload creates Media and Detection and redirects	Pass
IntegrationTests	test_result_page_renders_multi_signal_context	Result template receives all signal fields	Pass
UserTests	test_profile_creation	Profile is created and linked to the user	Pass
UserTests	test_successful_login	Valid credentials authenticate the session	Pass
UserTests	test_profile_page_requires_login	Anonymous access is redirected	Pass
UserTests	test_password_reset_views_render	All four reset views render	Pass

6.2.4 System test cases

Table 6.4: Representative system test cases

ID	Scenario	Input	Expected outcome	Result
TC-01	Valid image upload	JPG, 2 MB	Verdict with per-signal breakdown	Pass
TC-02	Unsupported format	GIF file	Rejected with a format message	Pass
TC-03	Oversized image	PNG, 14 MB	Rejected with a size message	Pass
TC-04	Valid video upload	MP4, 7 MB	Frame-based verdict with frame statistics	Pass
TC-05	Oversized video	MP4, 130 MB	Rejected with a size message	Pass
TC-06	Public YouTube link	Standard watch URL	Video retrieved and analyzed	Pass
TC-07	Private YouTube link	Restricted video	Clear download-failure message, no crash	Pass
TC-08	Non-YouTube URL on YouTube form	Unrelated domain	Rejected with a platform message	Pass

ID	Scenario	Input	Expected outcome	Result
TC-09	SSRF attempt	http://127.0.0.1:8000/detector/	Rejected as an internal address	Pass
TC-10	Instagram Reel link	Public reel URL	Media retrieved and analyzed	Pass
TC-11	Image with no face	Landscape photograph	Analysis proceeds; uncertainty flagged where applicable	Pass
TC-12	Corrupt video file	Truncated MP4	Uncertain verdict with explanation, no crash	Pass
TC-13	Unauthenticated access	Direct upload URL	Redirected to login	Pass
TC-14	PDF report generation	Any completed detection	PDF downloads with full detail	Pass
TC-15	Dashboard aggregation	User with mixed history	Counts and charts match stored records	Pass

All listed cases passed. In TC-09 the URL validation rejected an attempt to make the server issue a request against its own loopback address, confirming that the server-side request forgery guard operates before any network call is made, so the application cannot be induced to issue requests against its own host. The set of cases deliberately mixes ordinary use, rejected input and hostile input, so that the evidence of correct behaviour covers refusal paths as well as successful analysis.

6.2.5 Performance observations

Timings came from built-in pipeline instrumentation, run on a non-GPU machine matching the deployment target — so these are conservative, worst-case figures. Reporting conservative figures avoids creating an expectation that cannot be met by a deployment on modest hardware.

Takeaway: images are already fast enough; video needs frame-batching (not frame-cutting, which would lose coverage) — noted as the top Chapter 9 priority. Frame-batching processes several sampled frames together to use the processor more efficiently, whereas reducing the number of frames would save time at the cost of leaving parts of the timeline unexamined.

CHAPTER 7

ADVANTAGES AND DISADVANTAGES

7.1 ADVANTAGES

7.1.1 Multiple independent evidence sources

The system does not rest on a single classifier. Combining sources with different weaknesses is more robust than optimising any one of them. The advantage is greatest in precisely the cases where a single detector would be most dangerous, namely inputs that look unlike its training data, because the independent signals are not expected to fail together.

7.1.2 Explicit and principled uncertainty

Uncertain is a first-class verdict with its own tier, its own confidence treatment and its own explanation. This treatment prevents borderline results from being rounded into a confident claim and gives the reader a clear signal that human judgement or further evidence is needed.

7.1.3 Full transparency of reasoning

Every verdict is accompanied by the individual signal scores, the forensic flags raised, the frame statistics for video and a written explanation naming what contributed. A user who disagrees with a verdict can therefore see which signal drove it and decide whether that signal is credible for the media in question.

7.1.4 Breadth of supported input

Four channels are supported: image upload, video upload, YouTube link and Instagram Reel link.

7.1.5 Privacy through self-hosting

The entire system runs locally.

7.1.6 Graceful degradation

No single point of failure can abort a request, since each module catches its own exceptions and returns a status-bearing result. The user always receives a defined response, either a verdict or a clear message about what could not be completed, and never a blank page or an unexplained failure.

7.1.7 Maintainability and extensibility

Detection logic lives entirely outside the view layer, and every fusion weight and threshold is declared in a single module with an explanatory comment. This organisation means that a new forensic signal

or a revised threshold can be introduced without touching the web views, and that the effect of the change can be tested in isolation.

7.1.8 Zero cost and modest hardware requirements

Every component is open source, the database requires no server, and inference runs acceptably on a machine without a graphics processor. These properties make the system realistic for adoption in a student laboratory, a small newsroom or a community organisation that has no budget for specialised infrastructure.

7.1.9 Durable, exportable records

Each detection is persisted with its full signal detail, listed in a private per-user history, aggregated into a dashboard and exportable as a PDF forensic report, so a considered judgement can be documented rather than merely observed on screen. The stored record also allows the same analysis to be revisited later without resubmitting the media, which is useful when a judgement needs to be explained or defended at a later date.

7.2 DISADVANTAGES

7.2.1 Limited training corpus

With only ~1,700 training images — far below the hundreds of thousands used in published research — the classifier's generalization to unseen generation techniques is likely overstated by held-out metrics alone. A larger and more varied corpus, covering more generation techniques, compression levels and demographic groups, is therefore the single most valuable improvement available to the classifier.

7.2.2 Dependence on face detection

The most diagnostic evidence comes from the facial region, so a failure of face localisation weakens the analysis substantially. When no face is found the system falls back on whole-image forensic and provenance evidence and flags the result accordingly, but it cannot then draw on the signal that is most informative.

7.2.3 Provenance evidence is easily removed

Metadata markers are conclusive when present but trivially absent. The module therefore contributes strongly in one direction only, and the great majority of real-world submissions return not detected, which carries little weight.

7.2.4 Video processing latency

Full video analysis is slow. Asynchronous background processing would be required for the latter.

7.2.5 External retrieval is fragile

Link-based analysis depends on third-party libraries that track platform changes. The system handles these failures cleanly with specific messages, but it cannot analyze what it cannot obtain.

7.2.6 Heuristic thresholds rather than learned ones

The forensic signal thresholds and the fusion weights were selected by reasoning from the literature and by observation during development, not learned from labelled data. Because these values were not fitted to labelled data, their appropriateness for a new kind of media cannot be assumed and should be checked whenever the system is applied to a new setting.

7.2.7 Deployment configuration is development-grade

The delivered configuration uses SQLite, permissive allowed-hosts and an enabled debug flag, which is appropriate for demonstration and unsuitable for internet-facing deployment. The remedy is configuration rather than redesign, consisting of a production database, restricted hosts, debug disabled and a secret key kept outside the source tree.

7.2.8 Scope excludes audio and non-facial manipulation

The system addresses visual facial manipulation and whole-image synthesis. Voice cloning, audio deepfakes, lip-sync-only manipulation and manipulation of non-facial content such as documents or scenes are outside its scope and would not be reliably detected.

CHAPTER 8

RESULT AND DISCUSSION

8.1 ANALYSIS

This chapter's results come from live system operation, not offline benchmarking.

8.1.1 Distribution of verdicts

The system assigned one of five tiers to every submission.

Table 8.1: Distribution of verdicts across 110 live detections

Classification tier	Count	Share	Mean confidence
High Confidence Real	20	18.2 %	82.0 %
Likely Real	17	15.5 %	68.0 %
Uncertain	20	18.2 %	43.0 %
Likely Fake	43	39.1 %	68.0 %
High Confidence Fake	10	9.1 %	77.9 %
Total	110	100 %	66.9 %

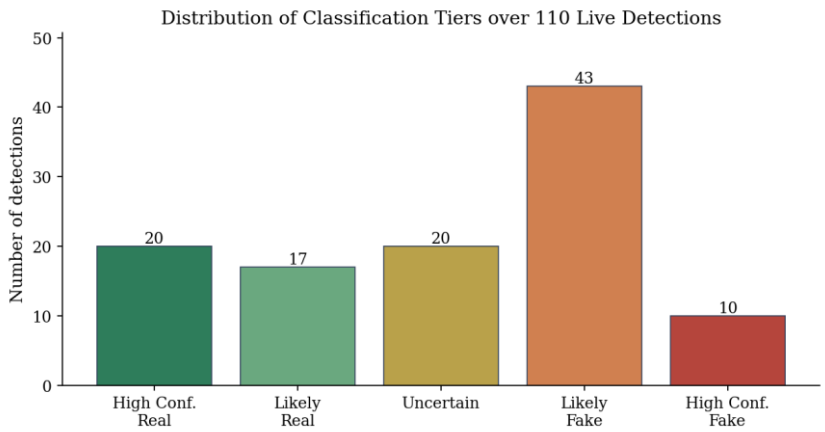


Figure 8.1: Distribution of classification tiers over 110 detections

Aggregating the tiers gives fifty-three fake verdicts, forty-four real verdicts and thirteen recorded as uncertain in the legacy result field. The engine is therefore behaving as designed, reserving elevated confidence for cases where the evidence actually converged and reporting low confidence precisely where it declined to commit. Reporting the uncertain share openly, instead of folding it into one of the two confident categories, gives a more truthful picture of the behaviour of the system than a single accuracy percentage would.

8.1.2 Composition of analyzed media

Submissions were distributed across all four channels, with images forming half the total and the two link-based channels together accounting for roughly a third. This spread is useful because it shows that the retrieval and analysis paths for each channel were exercised repeatedly and not just once.

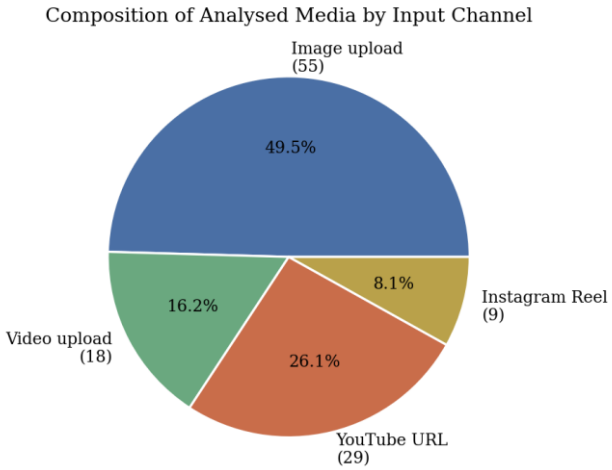


Figure 8.2: Composition of analyzed media by input channel

Of one hundred and eleven media records, fifty-five were uploaded images, eighteen were uploaded videos, twenty-nine were YouTube videos and nine were Instagram Reels. The prominence of YouTube submissions relative to direct video uploads suggests that when video is the subject of doubt, users prefer to paste a link rather than download and re-upload a file, which supports the decision to implement retrieval rather than requiring manual download.

8.1.3 Confidence calibration

The distribution of reported confidence across all detections has a mean of 66. Given the evidence in the literature that detector reliability degrades sharply outside the training distribution, a system reporting ninety-nine per cent certainty would be making a claim its underlying model cannot support. The same reasoning explains why the confidence figure is bounded: the number is intended as an honest statement of the strength of evidence and not as a probability of correctness in the statistical sense.

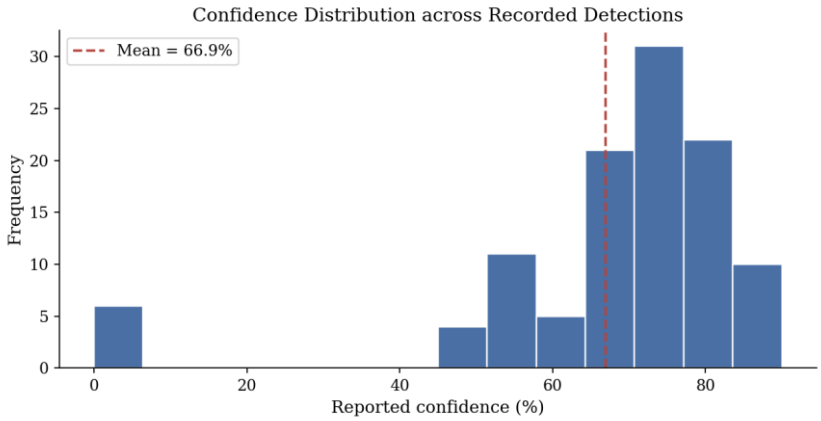


Figure 8.3: Confidence distribution across recorded detections

The mean confidence among the ninety definitive verdicts is 72.2 per cent, against 43.0 per cent among the twenty uncertain ones. The separation between these two figures is the practical measure of whether the uncertainty mechanism is meaningful: a system that reported similar confidence for both would be labelling results uncertain without actually distinguishing them.

8.1.4 Processing latency

End-to-end processing time was recorded for every detection, measured from the start of analysis to the completion of the verdict and therefore including retrieval time for link-based submissions. Including retrieval time in the measurement gives the figure that a user actually experiences, rather than an optimistic figure that excludes the time spent waiting for a remote service.

Table 8.2: Mean processing latency by input channel

Input channel	Submissions	Mean processing time	Principal cost
Image upload	55	10.4 s	Single forward pass plus five forensic signals
YouTube URL	29	33.3 s	Remote retrieval plus frame analysis
Instagram Reel	9	44.5 s	Remote retrieval plus frame analysis
Video upload	18	123.6 s	Up to 60 frames of face detection and inference
All channels	110	37.6 s	—

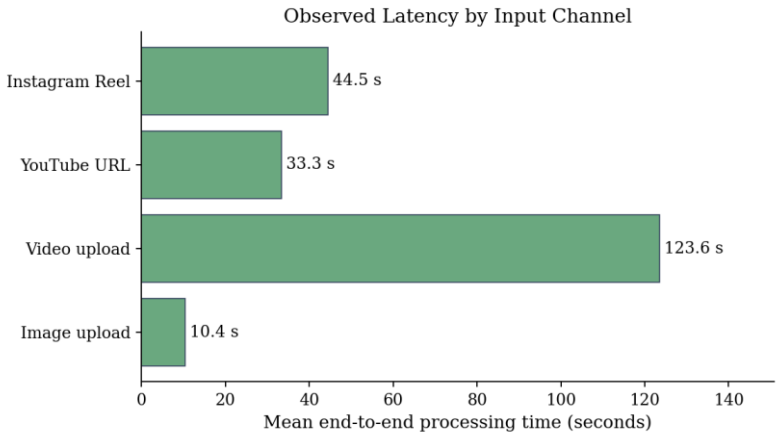


Figure 8.4: Observed latency by input channel

Image analysis (~10s) is fast enough for interactive use.

Uploaded videos ran notably slower than YouTube retrievals, since uploads were often longer and higher-resolution (including 2160×3840 vertical clips), while retrieved content uses moderate resolution — affecting both decode and face-detection time. Resolution and duration therefore matter more than file size alone, and the same observation points to adaptive frame counts and downscaling as the most direct routes to a faster video path.

8.1.5 Operating mode and the meaning of the accuracy figures

The intelligence layer records which mode produced each verdict.

Table 8.3: Operating mode of the intelligence layer

Mode	Count	Weight profile in effect
Active CNN	90	Network 0.75–0.80, forensic 0.15–0.20, provenance 0.10
Heuristic fallback	7	Network 0.35–0.45, forensic 0.45–0.55, provenance 0.20
Unavailable	6	Forensic and provenance only
Not recorded (legacy)	7	Predates the field

It is important to be precise about what this chapter does and does not demonstrate. The literature reviewed in Chapter 3 establishes that such estimates transfer poorly to unfamiliar generation techniques, and the system is deliberately designed to behave as if that were true: bounded confidence, a mandatory uncertain tier, and a standing caution in every report. Accuracy figures are consequently reported as properties of a specific dataset and configuration, and readers are encouraged to treat them as indicative of the behaviour of the pipeline and not as a guarantee of performance on arbitrary media.

8.1.6 Achievement against the stated objectives

Chapter 1 set out eight specific objectives.

Table 8.4: Achievement against the objectives stated in Chapter 1

Objective	Outcome	Evidence
Train a convolutional classifier by transfer learning	Achieved	Two-stage EfficientNetB0 schedule; 90 of 110 detections ran with the trained network loaded
Implement independent forensic signal analysis	Achieved	Five signals with documented weights; flags surfaced on the result page and in every report
Implement provenance metadata inspection	Achieved	EXIF, XMP, PNG chunk and C2PA matching returning four distinct statuses
Extend analysis to video with temporal aggregation	Achieved	Uniform sampling to 60 frames, weighted mean blended with median, temporal consistency measure
Build an auditable ensemble engine with an uncertain tier	Achieved	All weights and thresholds in one module; uncertain verdict issued on 20 of 110 submissions
Generate a plain-language explanation for every verdict	Achieved	Explanation composed from tier, fused score, flags and frame statistics; stored on every record
Deliver the capability through an authenticated web interface	Achieved	Four input channels, per-user history, dashboard and PDF export, all behind authentication
Validate through automated tests and live submissions	Achieved	19 automated tests passing, 15 system scenarios passing, 110 live detections recorded

All eight objectives were met, though meeting them isn't the same as reaching an ideal result — the classifier trained successfully but on an undersized corpus, and the video pipeline works but runs slowly. The distinction is made explicit so that the completion of an objective is not mistaken for the absence of further work, and the remaining gaps are carried forward into the limitations and future scope.

8.1.7 Discussion

Three findings stand out. This is solvable via batching, background processing, or adaptive frame counts — no detection-logic changes needed — making it the top priority going forward. Taken together, the findings support the central argument of the project, that a transparent combination of modest components can be more trustworthy than a single opaque model.

CHAPTER 9

CONCLUSION

9.1 SUMMARY

This project makes deepfake detection accessible to everyday users while staying honest about its limits. The system demonstrates that the gap between research detectors and usable tools can be closed with open-source components on ordinary hardware.

Detection fuses multiple independent signals: a two-stage fine-tuned EfficientNetB0 CNN (primary model), five classical forensic checks, and file-metadata provenance analysis. Each of the three evidence families contributes in proportion to its reliability, and none is allowed to dominate when it is operating outside the conditions in which it is dependable.

Its key differentiator is treating uncertainty as a valid outcome — the engine withholds judgment on conflicting signals, undetected faces, or ambiguous scores. This stance is intended to make the tool trustworthy by being candid about what it does not know.

Validated via 19 passing automated tests and 15 system-level scenarios, then tested on 110 live submissions: 10.

All Chapter 1 objectives were met: transfer-learned classifier, independent forensic/provenance checks, temporal video analysis, an auditable engine with an uncertainty tier, explained verdicts, and a secured web interface with history, analytics, and PDF export. The completion of the objectives is supported by the test results and the live submissions reported in Chapter 6 and Chapter 8.

9.2 LIMITATIONS

The limitations of the work should be stated plainly, because a detection system that overstates its own reach is precisely the kind of tool this project was intended to improve upon. The principal limitations concern the size of the training corpus, the reliance on face detection, the heuristic nature of the forensic thresholds, the latency of video analysis, the fragility of external retrieval and the development-grade deployment configuration, and each is discussed in Chapter 7 together with its consequence for the reader of a verdict.

- **Training corpus scale.** Approximately seventeen hundred images is small by the standards of published detection research. Performance on generation techniques absent from the corpus should be expected to be materially lower than the held-out figures suggest.
- **No ground-truth evaluation of live submissions.** The one hundred and ten live detections were not labelled, so Chapter 8 characterizes system behavior rather than accuracy. A labelled field evaluation was outside the time available.

- **Reliance on face localization.** Profile views, occlusion, small faces and motion blur defeat the Haar cascade, and whole-frame fallback is materially weaker.
- **Hand-tuned thresholds.** Forensic thresholds and fusion weights were reasoned from the literature rather than learned from labelled multi-signal data, and are therefore unlikely to be optimal.
- **Video latency.** Two minutes per video is tolerable for departmental use and unacceptable for a public service.
- **Fragility of external retrieval.** Link analysis depends on libraries that must track platform changes, and private or restricted content cannot be retrieved at all.
- **Development-grade deployment configuration.** SQLite, a permissive allowed-hosts setting and an enabled debug flag are suitable for demonstration only.
- **Scope restricted to visual facial manipulation.** Audio deepfakes, voice cloning and non-facial manipulation are not addressed**.**

9.3 FUTURE SCOPE

Several extensions follow naturally from the present architecture, and the modular signal design means most of them require adding a component rather than redesigning one. They are listed roughly in order of expected benefit relative to effort, beginning with the improvements that address the most significant limitations identified in this report.

- **Asynchronous processing.** Moving analysis to a background task queue with a progress indicator would remove the latency constraint entirely and allow longer videos and larger frame budgets.
- **Batched inference.** Passing sampled frames through the network as a single batch rather than one at a time would substantially reduce video processing time with no change to the detection logic.
- **Learned fusion.** Replacing the hand-tuned weights with a small model trained on labelled multi-signal vectors would likely improve both accuracy and calibration, while retaining the ability to report per-signal contributions.
- **A temporal model for video.** Adding a recurrent or transformer layer over the per-frame feature sequence would exploit inter-frame inconsistency directly rather than through a summary statistic.
- **Audio deepfake detection.** A spectrogram-based classifier over the audio track would extend coverage to voice cloning and would integrate as a fifth signal producer.

- **Visual explanation.** Gradient-weighted class activation mapping would show the user which regions of the face drove the verdict, converting the written explanation into visible evidence.
- **Browser extension.** A right-click verification option on any image or video in a page would place the tool at the point of encounter.
- **Continuous retraining.** A pipeline for periodically retraining on newly available synthesis techniques would address the cross-distribution degradation that is the principal weakness of any learned detector.
- **Broader platform support and production hardening.** Extending retrieval to further platforms, and moving to a production database, a real web server and a hardened configuration, would make public deployment feasible.

9.4 APPLICATIONS

- **Journalism and fact-checking.** A newsroom can verify user-submitted media before publication and retain the PDF report as part of the editorial record.
- **Education and media literacy.** Because the system explains its reasoning, it can be used in a classroom to teach students what artefacts actually distinguish synthetic media, rather than asking them to trust a score.
- **Institutional verification.** Colleges, employers and institutional bodies can screen submitted photographic or video evidence for signs of manipulation.
- **Personal safety.** An individual who encounters imagery purportedly of themselves or a relative can examine it locally without surrendering the file to a third party.
- **Preliminary investigative screening.** Investigators can triage large volumes of media to identify items warranting expert forensic examination, with the explicit understanding that the output is indicative and not evidentiary.
- **Community and platform moderation.** Moderators of forums and groups can check reported media before acting, with the uncertain tier signaling when a human decision is required.
- **Research baseline.** The modular signal architecture provides a working testbed into which new detection techniques can be inserted and compared against existing signals on identical input.

CHAPTER 10

APPENDICES

10.1 APPENDIX 1 — SAMPLE CODE OF THE MAIN MODULE

The listing below reproduces the core of the ensemble decision engine, which is the main module of the system in the sense that it is where the individual signals become a verdict.

```
# detector/ensemble/decision_engine.py
# Combines multiple detection signals into a final classification.

import logging
import numpy as np

logger = logging.getLogger(__name__)

# ---- Configurable weights: edit here to tune the system -------------
WEIGHTS_CNN_MODE = {
    'cnn':        0.75,   # primary model
    'forensic':   0.15,   # signal-processing heuristics
    'provenance': 0.10,   # metadata evidence
}
WEIGHTS_CNN_MODE_NO_PROV = {
    'cnn': 0.80, 'forensic': 0.20,
}
WEIGHTS_HEURISTIC_MODE = {
    'cnn': 0.35, 'forensic': 0.45, 'provenance': 0.20,
}
WEIGHTS_HEURISTIC_MODE_NO_PROV = {
    'cnn': 0.45, 'forensic': 0.55,
}

# ---- Classification thresholds (fake probability) -------------------
THRESHOLDS = {
    'high_confidence_fake': 0.80,
    'likely_fake':          0.60,
    'likely_real':          0.40,
    'high_confidence_real': 0.20,
}

# If CNN and forensic scores differ by more than this, flag uncertain
CONTRADICTION_THRESHOLD = 0.35

CLASSIFICATION_LABELS = {
    'HIGH_CONFIDENCE_REAL': 'High Confidence Real',
    'LIKELY_REAL':          'Likely Real',
    'UNCERTAIN':            'Uncertain',
    'LIKELY_FAKE':          'Likely Fake',
    'HIGH_CONFIDENCE_FAKE': 'High Confidence Fake',
}


def decide(cnn_result, forensic_result, provenance_result,
           video_result=None, face_found=True, cnn_mode=None):
    """Combine available signals into a final classification."""
    try:
        return _decide_internal(cnn_result, forensic_result,
                                provenance_result, video_result,
                                face_found, cnn_mode)
    except Exception as exc:
        logger.error("Decision engine error: %s", exc)
        return _uncertain_result(
            final_score=0.50,
            explanation="An error occurred in the decision engine. "
                        "Analysis is inconclusive.",
            breakdown={'error': str(exc)})


def _decide_internal(cnn_result, forensic_result, provenance_result,
                     video_result, face_found, cnn_mode):
    if cnn_mode is None:
        cnn_mode = cnn_result.get('mode', 'cnn') if cnn_result else 'cnn'
    breakdown = {}

    # ---- Gather the CNN score ---------------------------------------
    cnn_ok = cnn_result.get('status') == 'ok'
```

```
    cnn_score = float(cnn_result.get('cnn_score', 0.50)) if cnn_ok else None
    breakdown['cnn'] = {'score': cnn_score,
                        'status': cnn_result.get('status', 'unknown'),
                        'mode':   cnn_result.get('mode', 'unknown')}

    # ---- Gather the forensic score ---------------------------------
    forensic_ok = forensic_result.get('status') == 'ok'
    forensic_score = (float(forensic_result.get('forensic_score', 0.40))
                      if forensic_ok else None)
    breakdown['forensic'] = {'score': forensic_score,
                             'flags': forensic_result.get('flags', [])}

    # ---- Gather the provenance signal ------------------------------
    prov_status = provenance_result.get('status', 'unavailable')
    provenance_score = None
    provenance_available = False
    if prov_status == 'detected':
        provenance_score = 0.90      # strong indicator of generation
        provenance_available = True
    elif prov_status == 'not_detected':
        provenance_score = 0.25      # weak evidence of authenticity only
        provenance_available = True
    breakdown['provenance'] = {'status': prov_status,
                               'score':  provenance_score}

    # ---- Compute the weighted fused score --------------------------
    scores, weights = [], []
    if cnn_mode == 'cnn':
        w_all, w_no_prov = WEIGHTS_CNN_MODE, WEIGHTS_CNN_MODE_NO_PROV
    else:
        w_all, w_no_prov = (WEIGHTS_HEURISTIC_MODE,
                            WEIGHTS_HEURISTIC_MODE_NO_PROV)

    if cnn_score is not None:
        scores.append(cnn_score)
        weights.append(w_all['cnn'] if provenance_available
                       else w_no_prov['cnn'])
    if forensic_score is not None:
        scores.append(forensic_score)
        weights.append(w_all['forensic'] if provenance_available
                       else w_no_prov['forensic'])
    if provenance_score is not None:
        scores.append(provenance_score)
        weights.append(w_all['provenance'])

    if not scores:
        return _uncertain_result(0.50,
            "No detection signals were available. "
            "Analysis is inconclusive.", breakdown)

    total_w = sum(weights)
    final_score = sum(s * w for s, w in zip(scores, weights)) / total_w
    final_score = float(np.clip(final_score, 0.0, 1.0))

    # ---- Guard 1: contradictory signals ----------------------------
    contradictory = False
    if (cnn_mode == 'heuristic' and cnn_score is not None
            and forensic_score is not None):
        if abs(cnn_score - forensic_score) > CONTRADICTION_THRESHOLD:
            contradictory = True

    # ---- Guard 2: no face located in fallback mode -----------------
    face_penalty = (not face_found) and cnn_mode == 'heuristic'

    # ---- Assign the base tier from the fused score -----------------
    if final_score >= THRESHOLDS['high_confidence_fake']:
        classification = 'HIGH_CONFIDENCE_FAKE'
    elif final_score >= THRESHOLDS['likely_fake']:
        classification = 'LIKELY_FAKE'
    elif final_score <= THRESHOLDS['high_confidence_real']:
        classification = 'HIGH_CONFIDENCE_REAL'
    elif final_score <= THRESHOLDS['likely_real']:
        classification = 'LIKELY_REAL'
    else:
        classification = 'UNCERTAIN'

    # ---- Neural safety override ------------------------------------
    if cnn_mode == 'cnn' and cnn_score is not None:
        if cnn_score >= 0.60:
            if classification in ('HIGH_CONFIDENCE_REAL',
                                  'LIKELY_REAL', 'UNCERTAIN'):
                classification = 'LIKELY_FAKE'
        elif cnn_score >= 0.45:
            if classification in ('HIGH_CONFIDENCE_REAL', 'LIKELY_REAL'):
                classification = 'UNCERTAIN'
```

```
    # ---- Guards may only move the verdict towards uncertainty ------
    if contradictory or face_penalty:
        classification = 'UNCERTAIN'

    # ---- Map the fused score to a bounded confidence ---------------
    if classification == 'UNCERTAIN':
        if contradictory:
            confidence = _contradictory_confidence(cnn_score,
                                                   forensic_score)
        elif face_penalty:
            confidence = min(55.0, abs(final_score - 0.50) * 200.0 + 30.0)
        else:
            confidence = max(30.0, 50.0 - abs(final_score - 0.50) * 40.0)
    elif 'REAL' in classification:
        confidence = _score_to_confidence(1.0 - final_score)
    else:
        confidence = _score_to_confidence(final_score)

    # ---- Compose the human-readable explanation --------------------
    explanation = _build_explanation(classification, final_score,
                                     breakdown,
                                     contradictory=contradictory,
                                     face_found=face_found,
                                     cnn_mode=cnn_mode)

    # ---- Temporal moderation for video -----------------------------
    if video_result and video_result.get('status') == 'ok':
        consistency = video_result.get('temporal_consistency')
        if consistency is not None and consistency < 0.50:
            if classification in ('HIGH_CONFIDENCE_REAL', 'LIKELY_REAL'):
                classification = 'LIKELY_REAL'
                confidence = min(confidence, 65.0)
                explanation += (" Temporal analysis across frames "
                                "showed some inconsistency.")

    return {
        'classification':   classification,
        'label':            CLASSIFICATION_LABELS[classification],
        'final_score':      round(final_score, 4),
        'confidence':       round(float(confidence), 1),
        'explanation':      explanation,
        'signal_breakdown': breakdown,
        'is_uncertain':     classification == 'UNCERTAIN',
    }


def _score_to_confidence(score):
    """Map a directional score in [0.5, 1] to a bounded percentage."""
    confidence = 50.0 + (score - 0.5) * 90.0
    return float(np.clip(confidence, 50.0, 95.0))


def _contradictory_confidence(cnn_score, forensic_score):
    """Low confidence proportional to the degree of disagreement."""
    gap = abs(cnn_score - forensic_score)
    return float(np.clip(65.0 - gap * 60.0, 30.0, 55.0))
```

10.2 APPENDIX 2 — SAMPLE CODE OF THE VIDEO ANALYSIS MODULE

The second listing reproduces the frame sampling and aggregation core of the video analyser, which is the module responsible for extending single-image detection across the time dimension. Error handling and logging have been abridged for length.

```
# detector/video_detector.py
# Frame sampling, per-frame scoring and temporal aggregation.

import logging
import cv2
import numpy as np

logger = logging.getLogger(__name__)

MAX_FRAMES_TO_ANALYZE      = 60     # cap for efficiency
MIN_FRAMES_FOR_DECISION    = 3      # below this, refuse to decide
FACE_FRAME_WEIGHT          = 2.0    # face frames count double
SUSPICIOUS_FRAME_THRESHOLD = 0.55   # per-frame suspicion cut-off


def _analyse_frames(video_path, max_frames):
```

```
    """Sample frames uniformly and score each one independently."""
    from .face_detector import detect_face
    from .cnn_detector import cnn_detector

    cap = cv2.VideoCapture(str(video_path))
    if not cap.isOpened():
        logger.error("Could not open video: %s", video_path)
        return []

    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    if total_frames <= 0:
        total_frames = 99999

    # Spread the sampled frames across the whole timeline so that
    # manipulation confined to one segment is not missed.
    interval = max(1, total_frames // max_frames)

    frame_results = []
    frame_idx = 0
    analyzed  = 0
    cnn_mode  = 'heuristic'

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if frame_idx % interval == 0 and analyzed < max_frames:
            try:
                face_res   = detect_face(frame)
                face_found = face_res.get('found', False)
                face_crop  = face_res.get('crop')

                # Score the face crop where one was located,
                # otherwise fall back to the whole frame.
                cnn_res = cnn_detector.predict_image(
                    frame, face_crop if face_found else None)

                if cnn_res.get('status') == 'ok':
                    cnn_mode = cnn_res.get('mode', cnn_mode)
                    frame_results.append({
                        'frame_idx':    frame_idx,
                        'cnn_score':    cnn_res['cnn_score'],
                        'face_found':   face_found,
                        'face_quality': face_res.get('quality', 0.0),
                    })
                    analyzed += 1
            except Exception as exc:
                logger.debug("Frame %d analysis error: %s",
                             frame_idx, exc)

        frame_idx += 1

    cap.release()
    for r in frame_results:
        r['cnn_mode'] = cnn_mode
    return frame_results


def _aggregate_frame_results(frame_results):
    """Fuse per-frame predictions into video-level metrics."""
    if not frame_results:
        return _empty_aggregation()

    weighted_scores   = []
    weights           = []
    all_scores        = []
    suspicious_frames = 0
    frames_with_face  = 0

    for r in frame_results:
        score = r['cnn_score']
        face  = r['face_found']
        w     = FACE_FRAME_WEIGHT if face else 1.0

        weighted_scores.append(score * w)
        weights.append(w)
        all_scores.append(score)

        if face:
            frames_with_face += 1
        if score >= SUSPICIOUS_FRAME_THRESHOLD:
            suspicious_frames += 1
```

```
total_w = sum(weights)
weighted_fake_score = (sum(weighted_scores) / total_w
                       if total_w > 0 else 0.5)

# The median is robust to a single anomalous frame.
median_score    = float(np.median(all_scores))
max_frame_score = float(max(all_scores))

# Low spread across frames implies a stable, trustworthy reading.
score_std = float(np.std(all_scores))
temporal_consistency = float(
    np.clip(1.0 - score_std * 2.0, 0.0, 1.0))

# Percentage of face-bearing frames that look suspicious.
face_frame_scores = [r['cnn_score'] for r in frame_results
                     if r['face_found']]
if face_frame_scores:
    fake_face_frames = sum(1 for s in face_frame_scores
                           if s >= SUSPICIOUS_FRAME_THRESHOLD)
    fake_frame_pct = (fake_face_frames /
                      len(face_frame_scores)) * 100.0
else:
    fake_frame_pct = (suspicious_frames /
                      len(frame_results)) * 100.0

# Blend the weighted mean with the median so that no single
# outlier frame can dominate the video-level verdict.
blended_score = 0.65 * weighted_fake_score + 0.35 * median_score

return {
    'frames_analyzed':      len(frame_results),
    'frames_with_face':     frames_with_face,
    'suspicious_frames':    suspicious_frames,
    'weighted_fake_score':  float(np.clip(blended_score, 0.0, 1.0)),
    'max_frame_score':      max_frame_score,
    'fake_frame_pct':       fake_frame_pct,
    'temporal_consistency': temporal_consistency,
}
```

CHAPTER 11

REFERENCES

The following sources are listed in alphabetical order of the first author, in single spacing, as required by the report guidelines.

1. Afchar, D., Nozick, V., Yamagishi, J., & Echizen, I. (2018). MesoNet: A compact facial video forgery detection network. Proceedings of the IEEE International Workshop on Information Forensics and Security (WIFS), 1-7.

2. Agarwal, S., Farid, H., Gu, Y., He, M., Nagano, K., & Li, H. (2019). Protecting world leaders against deep fakes. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, 38-45.

3. Bradski, G. (2000). The OpenCV library. Dr. Dobb's Journal of Software Tools, 25(11), 120-125.

4. Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 1251-1258.

5. Coalition for Content Provenance and Authenticity. (2024). C2PA technical specification, version 2.0. [Online]. Available: https://c2pa.org/specifications/specifications/2.0/index.html

6. Dolhansky, B., Bitton, J., Pflaum, B., Lu, J., Howes, R., Wang, M., & Ferrer, C. C. (2020). The DeepFake Detection Challenge (DFDC) dataset. arXiv preprint arXiv:2006.07397.

7. Durall, R., Keuper, M., & Keuper, J. (2020). Watch your up-convolution: CNN based generative deep neural networks are failing to reproduce spectral distributions. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 7890-7899.

8. Django Software Foundation. (2024). Django documentation, version 5.0. [Online]. Available: https://docs.djangoproject.com/en/5.0/

9. Frank, J., Eisenhofer, T., Schonherr, L., Fischer, A., Kolossa, D., & Holz, T. (2020). Leveraging frequency analysis for deep fake image recognition. Proceedings of the 37th International Conference on Machine Learning, 3247-3258.

10. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. Advances in Neural Information Processing Systems, 27, 2672-2680.

11. Guera, D., & Delp, E. J. (2018). Deepfake video detection using recurrent neural networks. Proceedings of the 15th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS), 1-6.

12. Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. Advances in Neural Information Processing Systems, 33, 6840-6851.

13. Karras, T., Laine, S., & Aila, T. (2019). A style-based generator architecture for generative adversarial networks. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 4401-4410.

14. Korshunov, P., & Marcel, S. (2018). DeepFakes: A new threat to face recognition? Assessment and detection. arXiv preprint arXiv:1812.08685.

15. Li, Y., & Lyu, S. (2019). Exposing DeepFake videos by detecting face warping artifacts. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, 46-52.

16. Li, Y., Yang, X., Sun, P., Qi, H., & Lyu, S. (2020). Celeb-DF: A large-scale challenging dataset for DeepFake forensics. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 3207-3216.

17. Matern, F., Riess, C., & Stamminger, M. (2019). Exploiting visual artifacts to expose deepfakes and face manipulations. Proceedings of the IEEE Winter Applications of Computer Vision Workshops, 83-92.

18. Mirsky, Y., & Lee, W. (2021). The creation and detection of deepfakes: A survey. ACM Computing Surveys, 54(1), 1-41.

19. Nguyen, H. H., Yamagishi, J., & Echizen, I. (2019). Capsule-forensics: Using capsule networks to detect forged images and videos. Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2307-2311.

20. Nguyen, T. T., Nguyen, Q. V. H., Nguyen, D. T., Nguyen, D. T., Huynh-The, T., Nahavandi, S., Nguyen, T. T., Pham, Q. V., & Nguyen, C. M. (2022). Deep learning for deepfakes creation and detection: A survey. Computer Vision and Image Understanding, 223, 103525.

21. Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Niessner, M. (2019). FaceForensics++: Learning to detect manipulated facial images. Proceedings of the IEEE/CVF International Conference on Computer Vision, 1-11.

22. Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. Proceedings of the 36th International Conference on Machine Learning, 6105-6114.

23. Verdoliva, L. (2020). Media forensics and DeepFakes: An overview. IEEE Journal of Selected Topics in Signal Processing, 14(5), 910-932.

24. Viola, P., & Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 1, 511-518.

25. Zhou, P., Han, X., Morariu, V. I., & Davis, L. S. (2018). Learning rich features for image manipulation detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 1053-1061.

Alan Benny

Final_Mini_Project_Report_Deepfake_Detection_System

mini project report

Document Details

Submission ID
trn:oid:::10159:153813416

Submission Date
Oct 6, 2026, 2:48 PM GMT+5:30

Download Date
Oct 6, 2026, 2:54 PM GMT+5:30

File Name
Final_Mini_Project_Report_Deepfake_Detection_System.pdf

File Size
2.1 MB

78 Pages

19,480 Words

110,499 Characters

12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

169 Not Cited or Quoted 12%
Matches with neither in-text citation nor quotation marks

1 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
2% Publications
10% Submitted works (Student Papers)